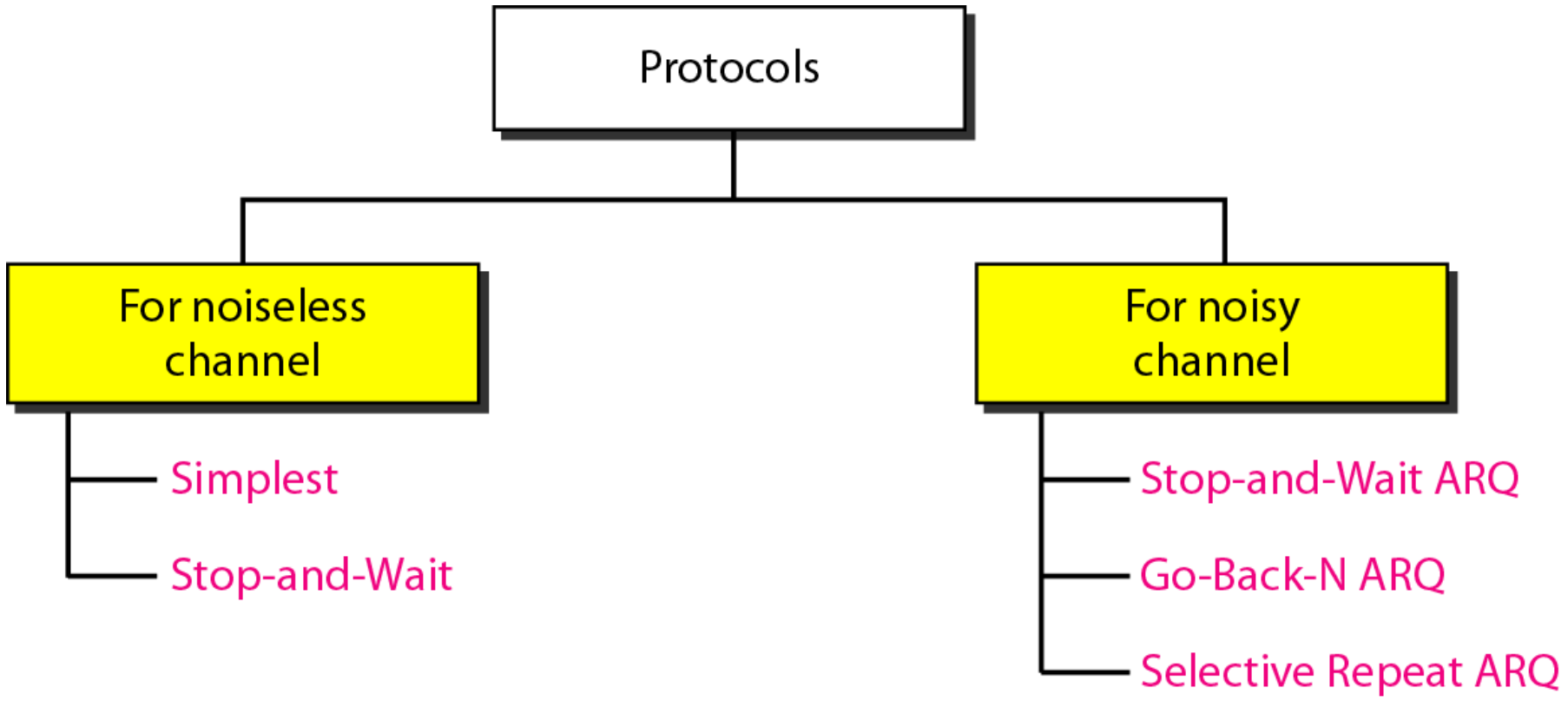
Flow and Error control Protocols

PROTOCOLS

- The data link layer can combine framing, flow control, and error control to achieve the delivery of data from one node to another.
- The protocols are normally implemented in software by using one of the common programming languages.
- Pseudocode a version of each protocol that concentrates mostly on the procedure instead of delving into the details of language rules.

Taxonomy of protocols discussed in this chapter



NOISELESS CHANNELS

- An ideal channel in which no frames are lost, duplicated, or corrupted.
- Two protocols for this type of channel.
  - Simplest Protocol
  - Stop-and-Wait Protocol

Finite State Machine

- The behavior of a data-link-layer protocol can be better shown as a finite state machine (FSM).
- An FSM is thought of as a machine with a finite number of states.
- The machine is always in one of the states until an event occurs.
- Each event is associated with two reactions: defining the list (possibly empty) of actions to be performed and determining the next state (which can be the same as the current state).
- One of the states must be defined as the initial state, the state in which the machine starts when it turns on.

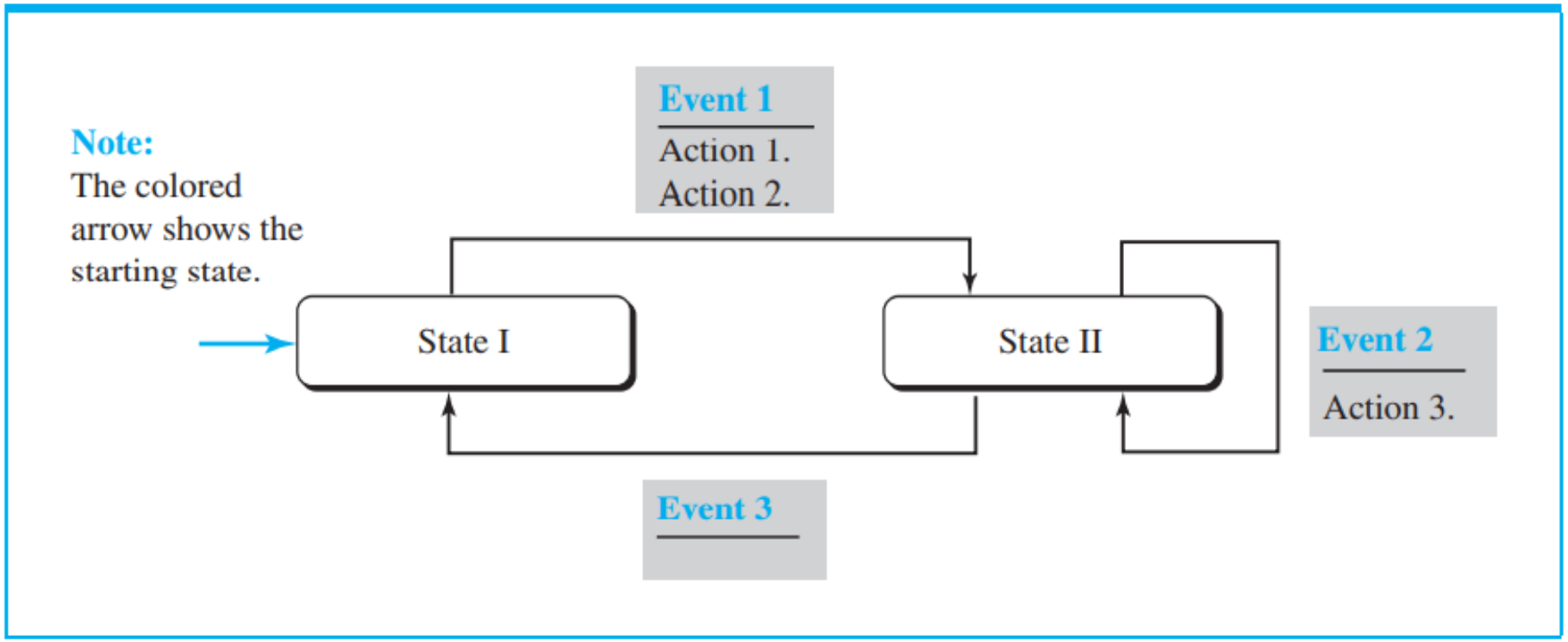
Contd...

- Rounded-corner rectangles to show states, colored text to show events, and regular black text to show actions.
- A horizontal line is used to separate the event from the actions, although later we replace the horizontal line with a slash.
- The arrow shows the movement to the next state.
- The figure shows a machine with three states.
- There are only three possible events and three possible actions.

Contd…

- The machine starts in state I. If event 1 occurs, the machine performs actions 1 and 2 and moves to state II.
- When the machine is in state II, two events may occur.
- If event 1 occurs, the machine performs action 3 and remains in the same state, state II.
- If event 3 occurs, the machine performs no action, but move to state I.

Connectionless and connection-oriented service represented as FSMs



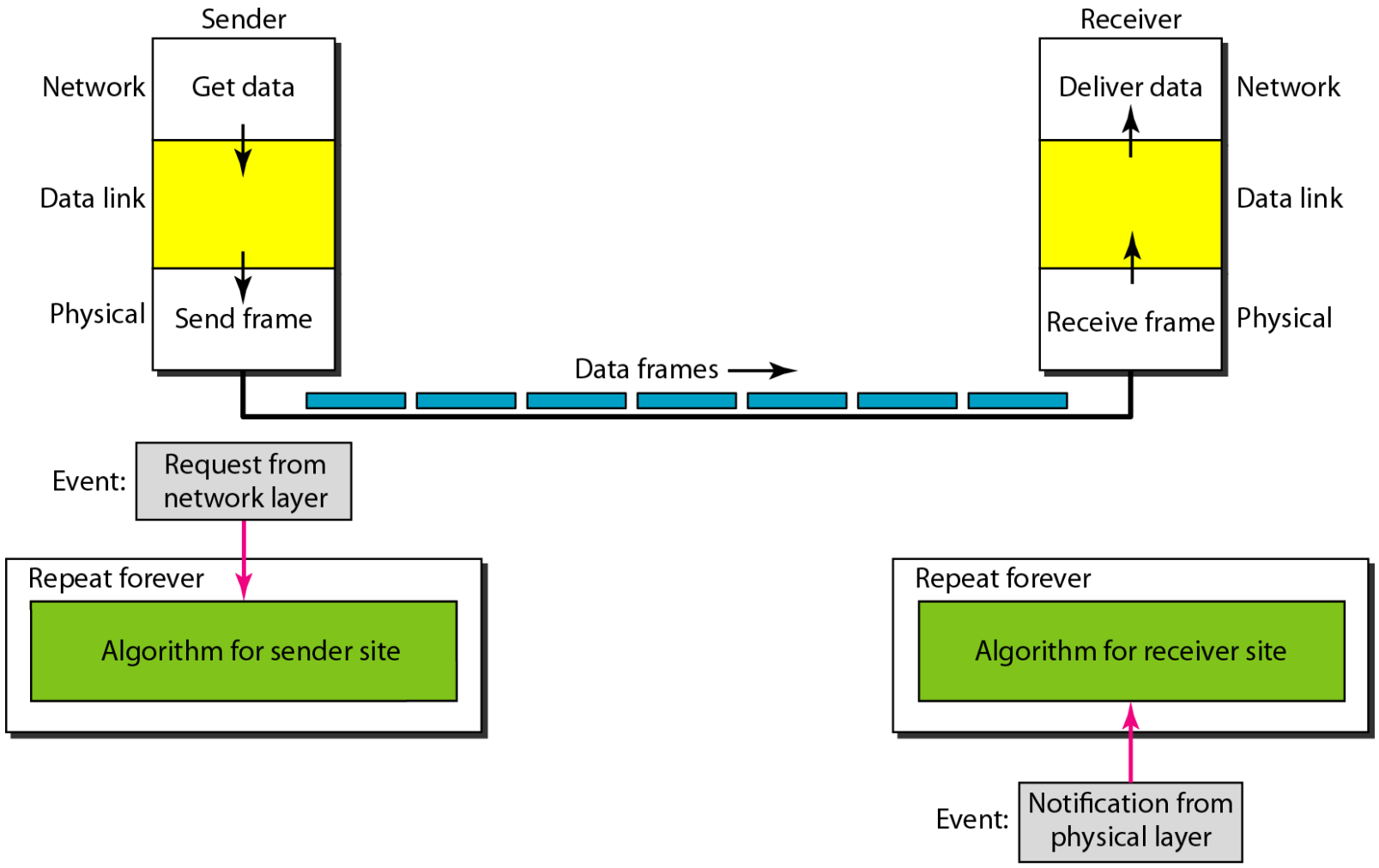
Simple Protocol

- A simple protocol with neither flow nor error control.
- The receiver can immediately handle any frame it receives.
- i.e.The receiver can never be overwhelmed with incoming frames.
- The data-link layer at the sender gets a packet from its network layer, makes a frame out of it, and sends the frame.

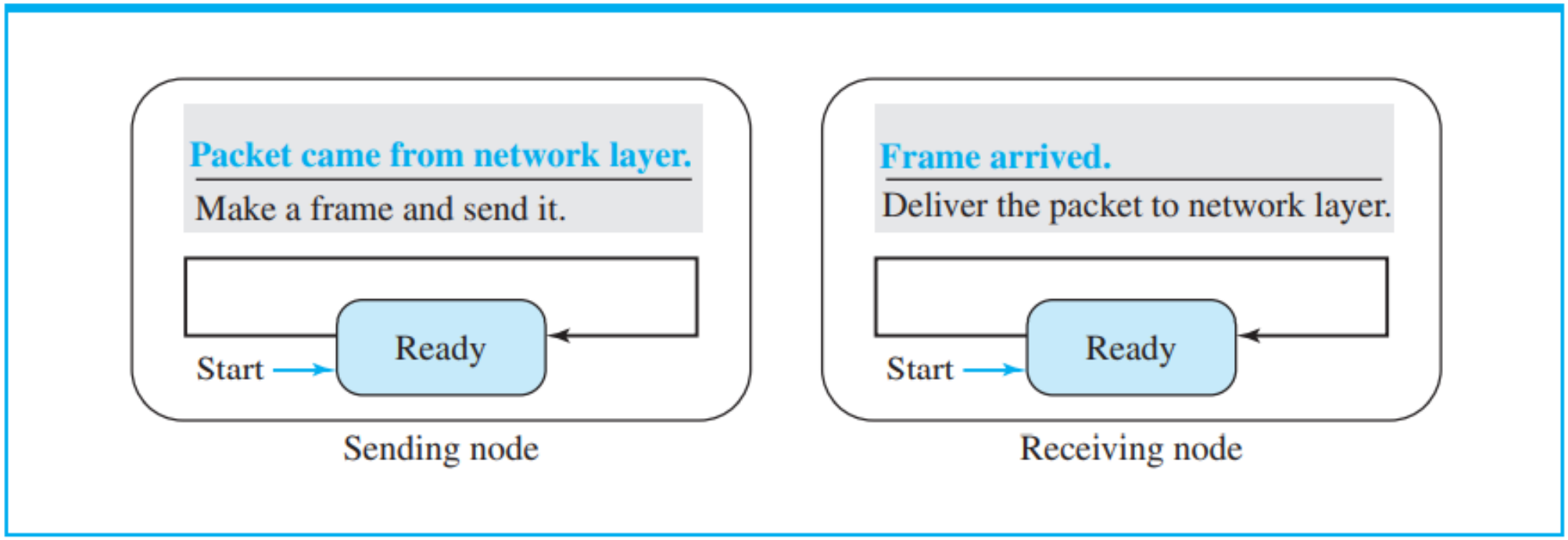
CONTD…

- The data-link layer at the receiver receives a frame from the link, extracts the packet from the frame, and delivers the packet to its network layer.
- The data-link layers of the sender and receiver provide transmission services for their network layers.

The design of the simplest protocol with no flow or error control



FSMs for the simple protocol



FSMs

- The sender side should not send a frame until its network layer has a message to send. The receiver site cannot deliver a message to its network layer until a frame arrives.
- Requirements using two FSMs. Each FSM has only one state, the ready state.
- The sending machine remains in the ready state until a request comes from the process in the network layer.

CONTD...

- When this event occurs, the sending machine encapsulates the message in a frame and sends it to the receiving machine.
- The receiving machine remains in the ready state until a frame arrives from the sending machine.
- When this event occurs, the receiving machine decapsulates the message out of the frame and delivers it to the process at the network layer.

Algorithm *Sender-site algorithm for the simplest protocol*

```
1  while(true)                          // Repeat forever
2  {
3    WaitForEvent();                    // Sleep until an event occurs
4    if(Event(RequestToSend))           //There is a packet to send
5    {
6       GetData();
7       MakeFrame();
8       SendFrame();                    //Send the frame
9    }
10 }
```

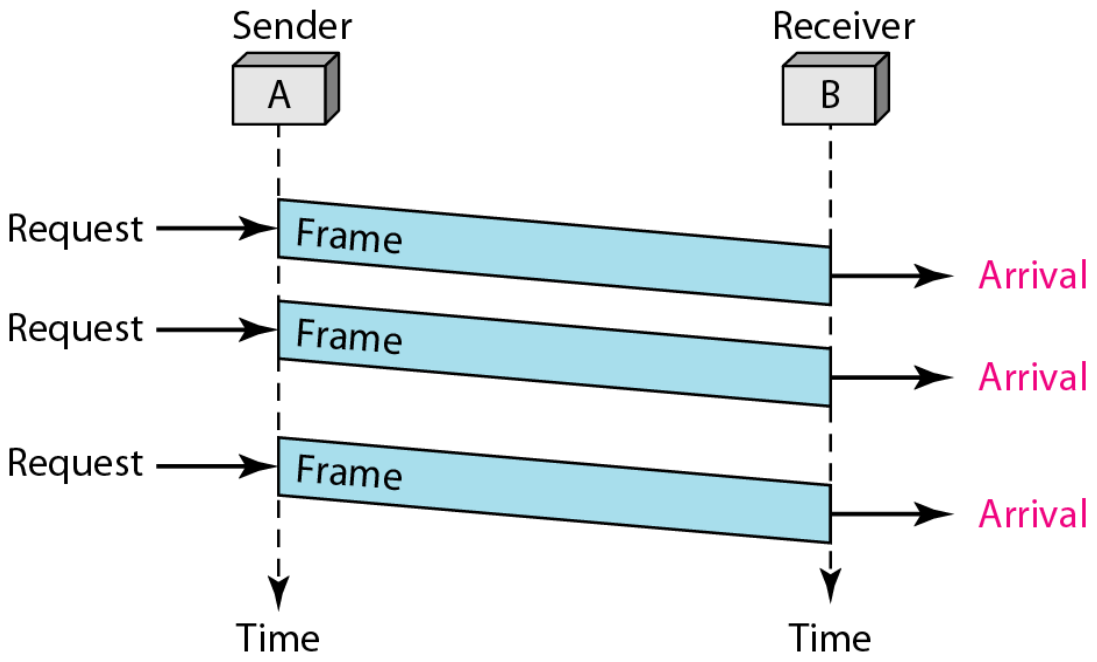
Algorithm *Receiver-site algorithm for the simplest protocol*

```
1  while(true)                              // Repeat forever
2  {
3    WaitForEvent();                        // Sleep until an event occurs
4    if(Event(ArrivalNotification)) //Data frame arrived
5    {
6       ReceiveFrame();
7       ExtractData();
8       DeliverData();                      //Deliver data to network layer
9    }
10 }
```

Example 11.1

- *Figure shows an example of communication using this protocol. It is very simple.*
- *The sender sends a sequence of frames without even thinking about the receiver.*
- *To send three frames, three events occur at the sender site and three events at the receiver site.*
- *The data frames are shown by tilted boxes; the height of the box defines the transmission time difference between the first bit and the last bit in the frame.*

Flow diagram for Example



Stop-and-Wait Protocol

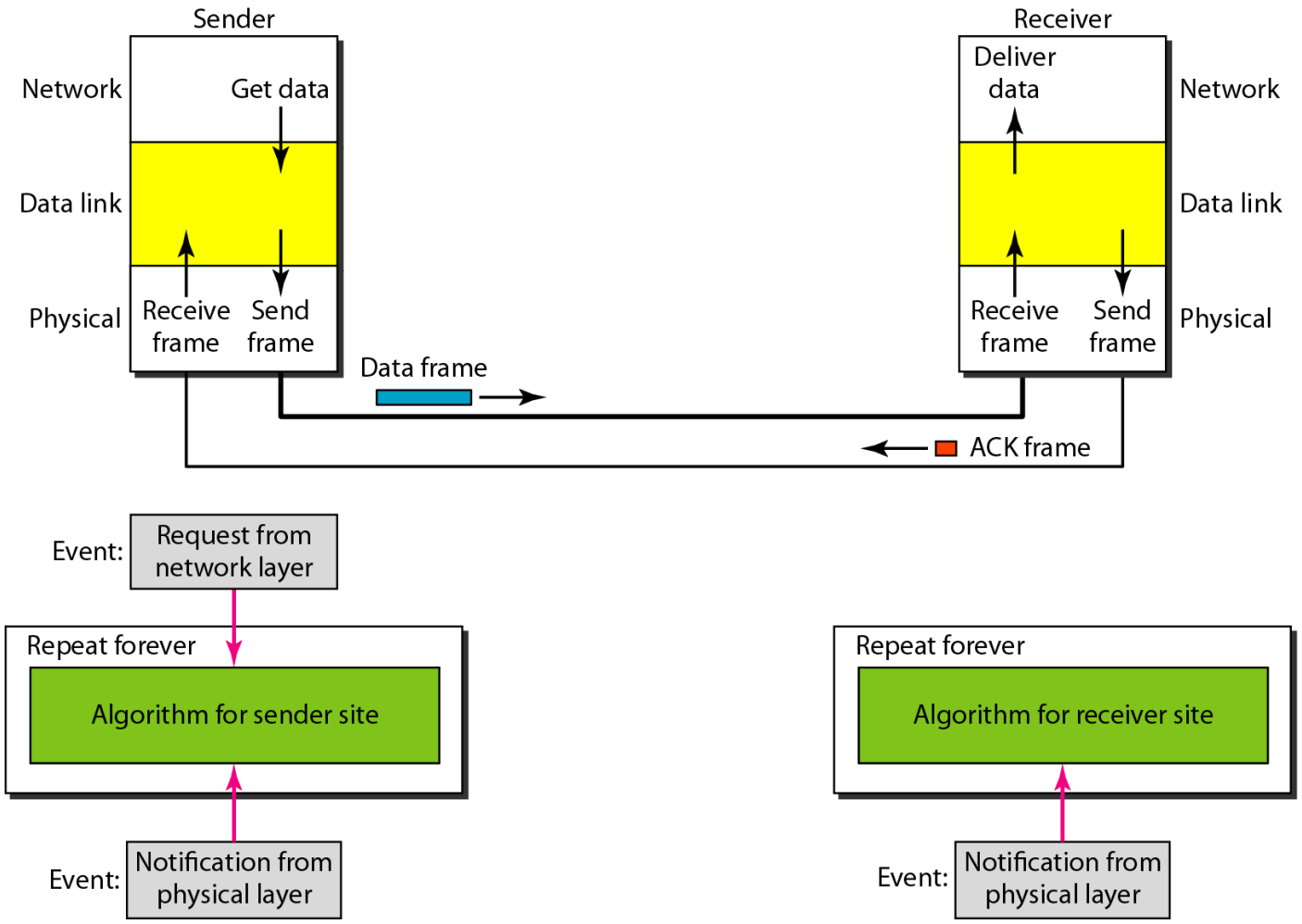
Stop-and-Wait Protocol

- In this protocol, the sender sends one frame at a time and waits for an acknowledgment before sending the next one.
- To detect corrupted frames, we need to add a CRC to each data frame.
- When a frame arrives at the receiver site, it is checked. If its CRC is incorrect, the frame is corrupted and silently discarded.
- The silence of the receiver is a signal for the sender that a frame was either corrupted or lost.
- Every time the sender sends a frame, it starts a timer.

Contd…

- If an acknowledgment arrives before the timer expires, the timer is stopped and the sender sends the next frame (if it has one to send).
- If the timer expires, the sender resends the previous frame, assuming that the frame was either lost or corrupted.
- This means that the sender needs to keep a copy of the frame until its acknowledgment arrives.
- The sender discards the copy and sends the next frame if it is ready.

Design of Stop-and-Wait Protocol



CONTD…

The FSMs for our primitive Stop-and-Wait protocol.

Sender States: The sender is initially in the ready state, but it can move between the ready and blocking state.

Ready State:

- When the sender is in this state, it is only waiting for a packet from the network layer.
- If a packet comes from the network layer, the sender creates a frame, saves a copy of the frame, starts the only timer and sends the frame.
- The sender then moves to the blocking state.

CONTD...

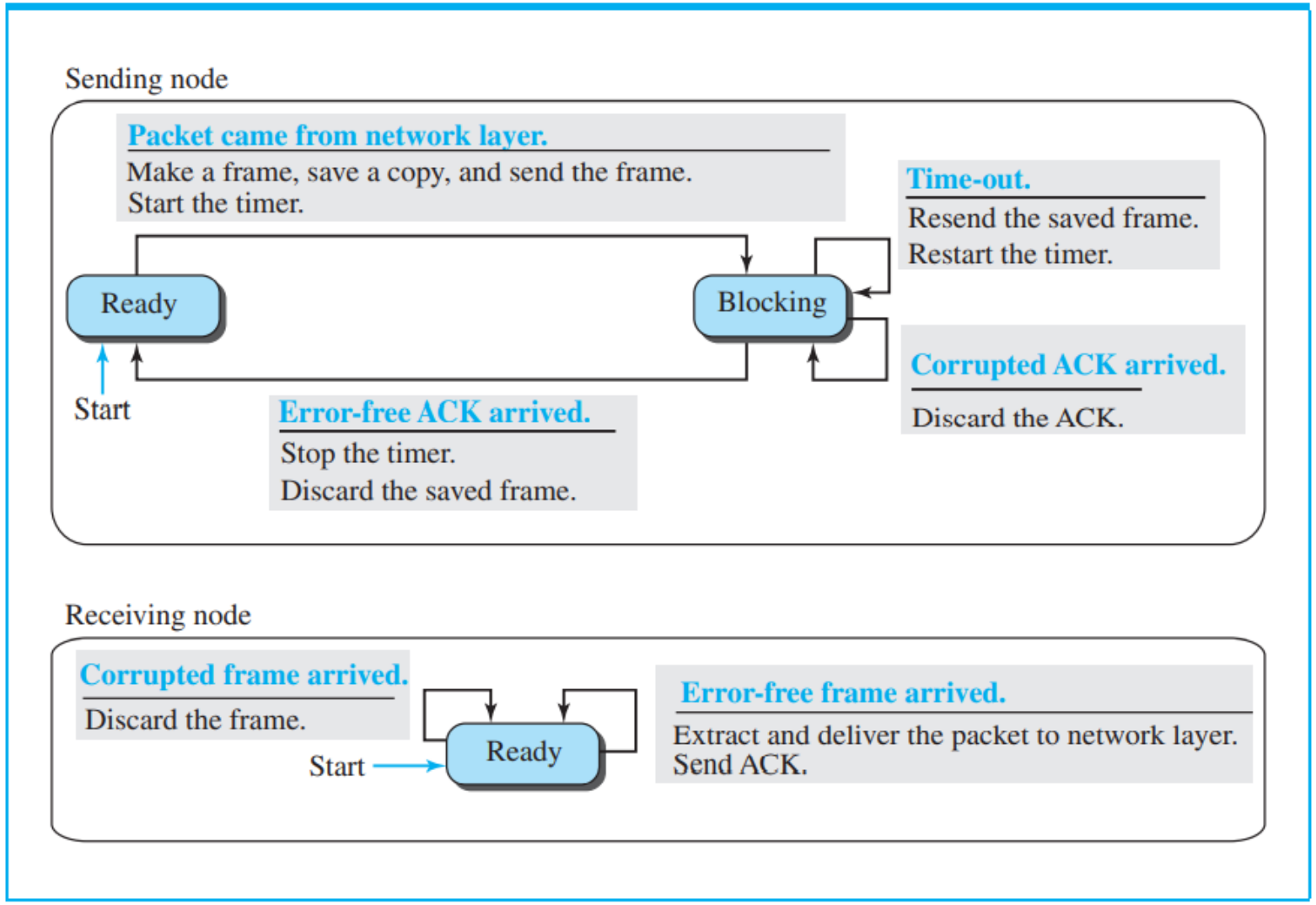
Blocking State:

a. If a time-out occurs, the sender resends the saved copy of the frame and restarts the timer.

b. If a corrupted ACK arrives, it is discarded.

c. If an error-free ACK arrives, the sender stops the timer and discards the saved copy of the frame. It then moves to the ready state.

FSM for the Stop-and-Wait protocol



CONTD…

Receiver:

The receiver is always in the ready state.

Two events may occur:

a. If an error-free frame arrives, the message in the frame is delivered to the network layer and an ACK is sent.

b. If a corrupted frame arrives, the frame is discarded.

Sequence and Acknowledgment Numbers

- Duplicate packets, as much as corrupted packets, need to be avoided.
- As an example, assume we are ordering some item online.
- If each packet defines the specification of an item to be ordered, duplicate packets mean ordering an item more than once.
- we need to add sequence numbers to the data frames and acknowledgment numbers to the ACK frames.

CONTD...

- Sequence numbers are 0, 1, 0, 1, 0, 1, . . . ; the acknowledgment numbers can also be 1, 0, 1, 0, 1, 0, …
- The sequence numbers start with 0, the acknowledgment numbers start with 1.
- An acknowledgment number always defines the sequence number of the next frame to receive.

Algorithm *Sender-site algorithm for Stop-and-Wait Protocol*

```
1  while(true)                          //Repeat forever
2  canSend = true                       //Allow the first frame to go
3  {
4    WaitForEvent();                    // Sleep until an event occurs
5    if(Event(RequestToSend) AND canSend)
6    {
7       GetData();
8       MakeFrame();
9       SendFrame();                    //Send the data frame
10      canSend = false;                //Cannot send until ACK arrives
11   }
12   WaitForEvent();                    // Sleep until an event occurs
13   if(Event(ArrivalNotification)      // An ACK has arrived
14    {
15      ReceiveFrame();                 //Receive the ACK frame
16      canSend = true;
17    }
18 }
```

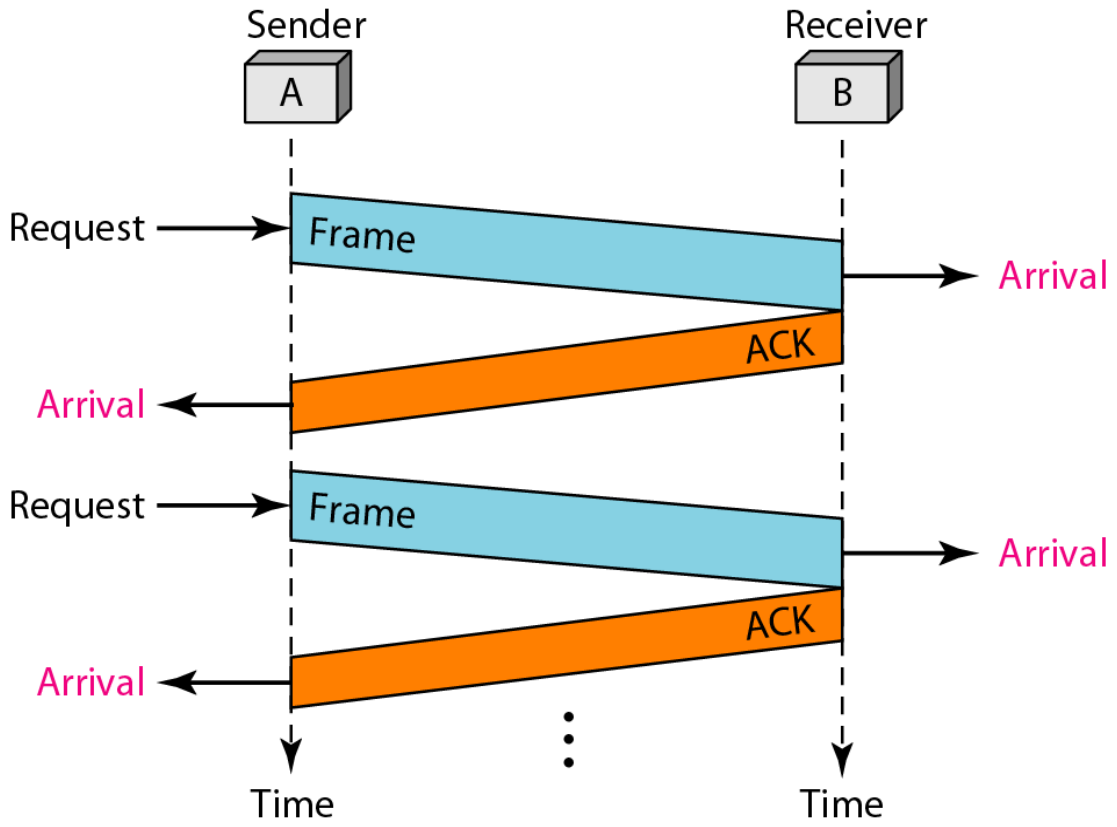
Algorithm *Receiver-site algorithm for Stop-and-Wait Protocol*

```
1  while(true)                              //Repeat forever
2  {
3    WaitForEvent();                        // Sleep until an event occurs
4    if(Event(ArrivalNotification))         //Data frame arrives
5    {
6       ReceiveFrame();
7       ExtractData();
8       Deliver(data);                      //Deliver data to network layer
9       SendFrame();                        //Send an ACK frame
10   }
11 }
```

Example 11.2

- *Figure shows an example of communication using this protocol. It is still very simple.*
- *The sender sends one frame and waits for feedback from the receiver.*
- *When the ACK arrives, the sender sends the next frame.*
- *Sending two frames in the protocol involves the sender in four events and the receiver in two events.*

Flow diagram for Example



NOISY CHANNELS

- *The Stop-and-Wait Protocol gives us an idea of how to add flow control to its predecessor, noiseless channels are nonexistent.*
- *Three protocols that use error control.*

✓ Stop-and-Wait Automatic Repeat Request
✓ Go-Back-N Automatic Repeat Request
✓ Selective Repeat Automatic Repeat Request

Stop and Wait ARQ

Note

Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.

Note

In Stop-and-Wait ARQ, we use sequence numbers to number the frames. The sequence numbers are based on modulo-2 arithmetic.

Note

In Stop-and-Wait ARQ, the acknowledgment number always announces in modulo-2 arithmetic the sequence number of the next frame expected.

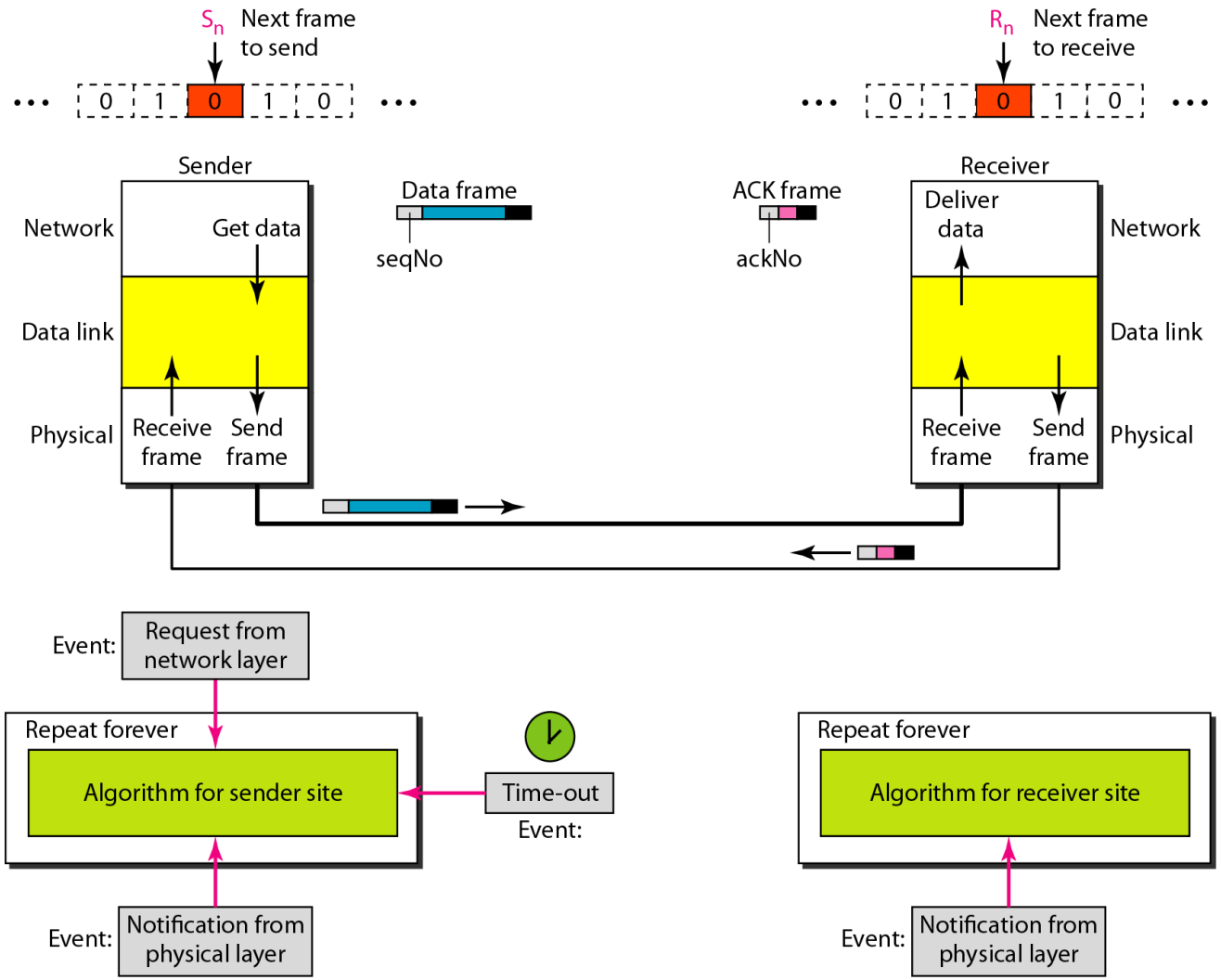
Stop-and-Wait ARQ Protocol

- It is the simplest flow and error control mechanism. A transmitter sends a frame then stops and waits for an acknowledgment.
- Stop-and-Wait ARQ has the following features:
  - ✓ The sending device keeps a copy of the sent frame transmitted until it receives an acknowledgment(ACK).
  - ✓ The sender starts a timer when it sends a frame. If an ACK is not received within an allocated time period, the sender resends it.

CONTD…

- ✓ Both frames and acknowledgment (ACK) are numbered alternately 0 and 1(two sequence number only).
- ✓ This numbering allows for identification of frames in case of duplicate transmission.

Design of the Stop-and-Wait ARQ Protocol



CONTD…

- The acknowledgment number defines the number of next expected frame. (frame 0 received ACK 1 is sent).
- A damage or lost frame treated by the same manner by the receiver.
- If the receiver detects an error in the received frame, or receives a frame out of order it simply discards the frame.

CONTD…

- The receiver send only positive ACK for frames received safe; it is silent about the frames damage or lost.
- The sender has a control variable S that holds the number of most recently sent frame (0 or 1). The receiver has control variable R, that holds the number of the next frame expected (0,or 1)

CONTD…

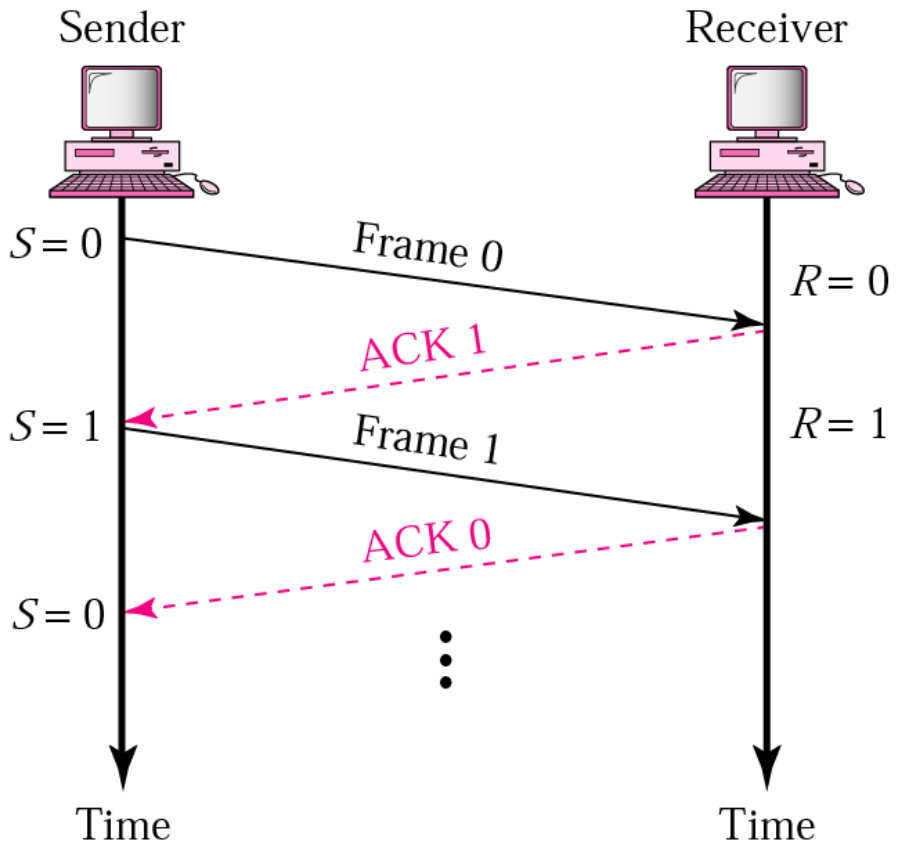
Cases of Operations:

- Normal operation
- The frame is lost
- The Acknowledgment (ACK) is lost
- The Ack is delayed

CONTD…

Normal operation

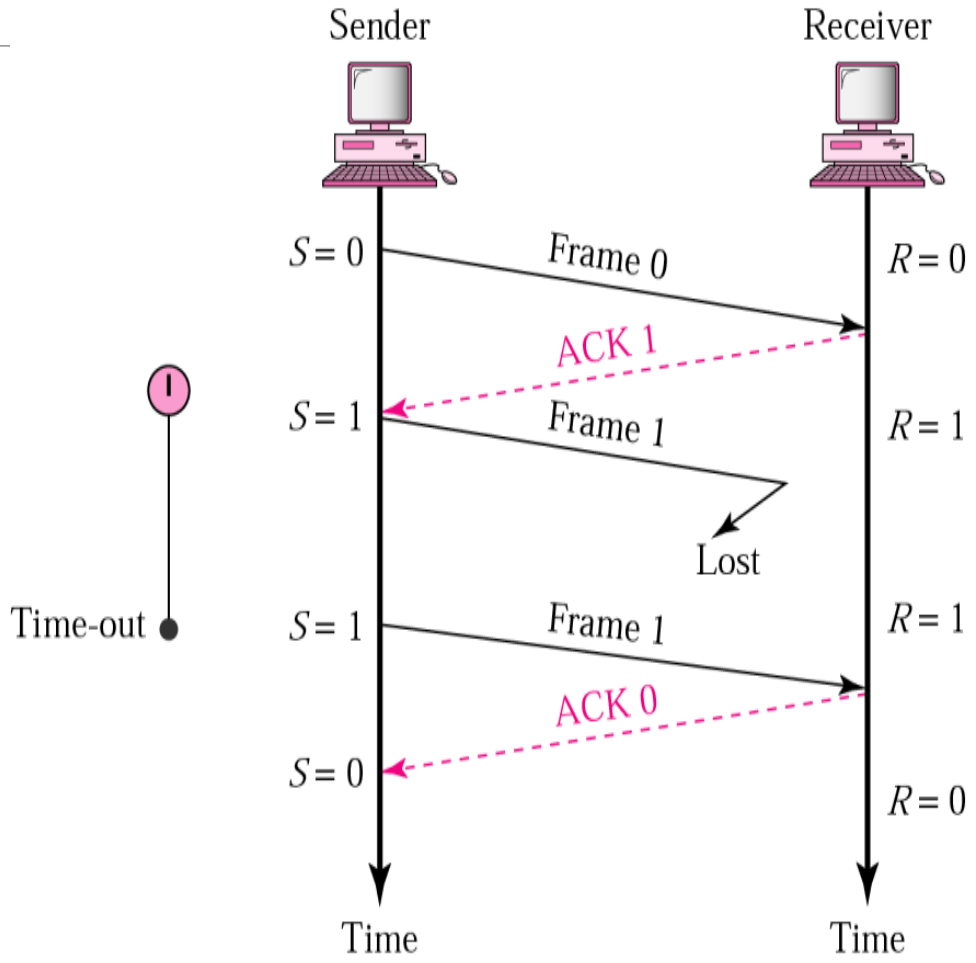
- The sender will not send the next frame until it is sure that the current one is correctly receive
- Sequence number is necessary to check for duplicated frames



CONTD…

Lost or damaged frame

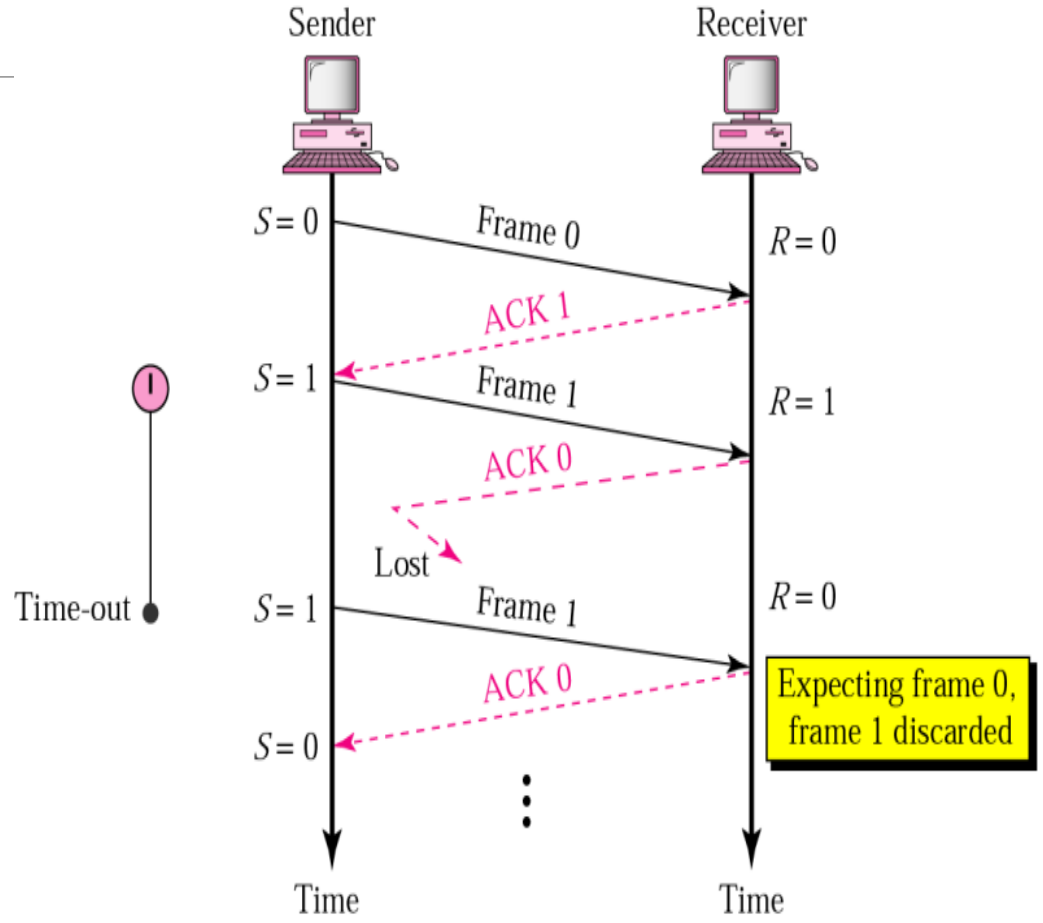
- A damage or lost frame treated by the same manner by the receiver.
- No ACK when frame is corrupted / duplicate



CONTD…

Lost ACK frame

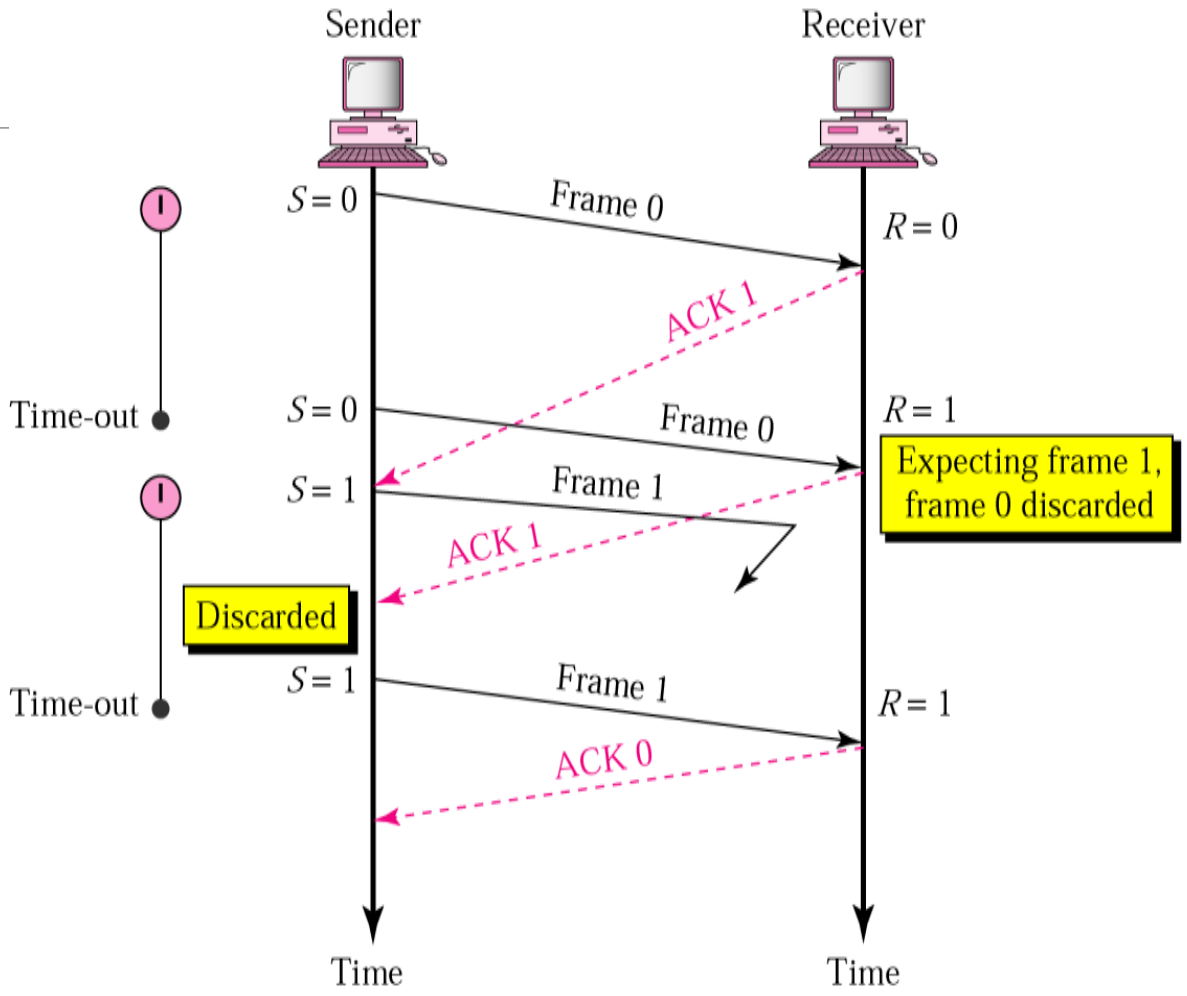
- Importance of frame numbering



CONTD...

Delayed ACK and lost frame

Importance of frame numbering



Note

In Stop and-Wait ARQ, numbering frames prevents the retaining of duplicate frames.

Note

Numbered acknowledgments are needed if an acknowledgment is delayed and the next frame is lost.

Piggybacking (Bidirectional transmission)

- Is a method to combine a data frame with an acknowledgment.
- It can save bandwidth because data frame and an ACK frame can combined into just one frame

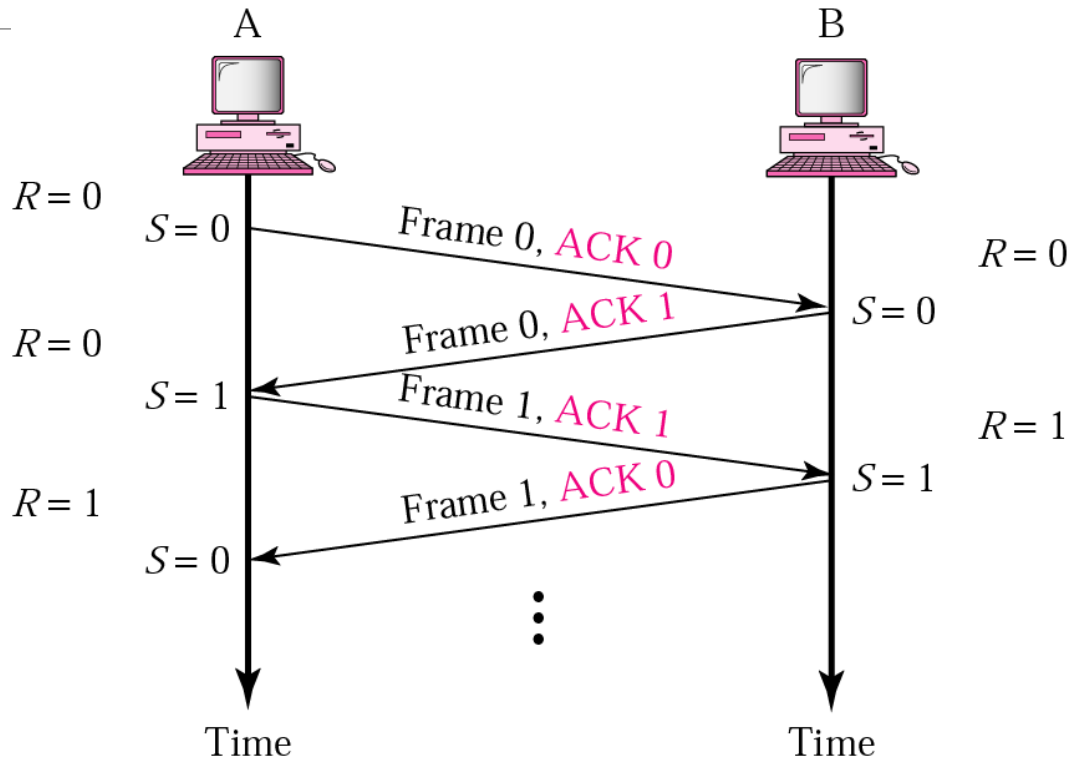
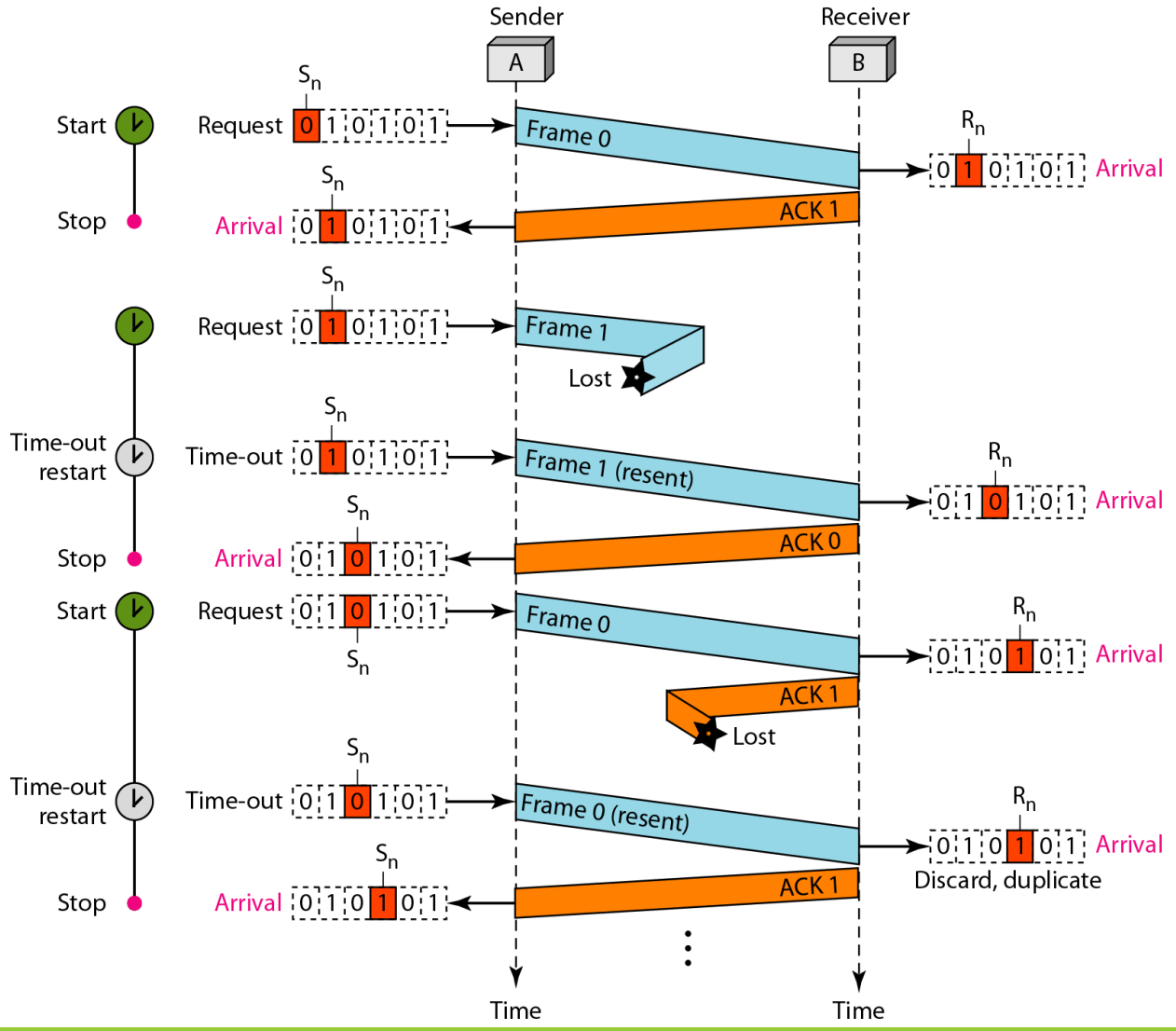


Figure shows an example of Stop-and-Wait ARQ.

- *Frame 0 is sent and acknowledged.*
- *Frame 1 is lost and resent after the time-out. The resent frame 1 is acknowledged and the timer stops.*
- *Frame 0 is sent and acknowledged, but the acknowledgment is lost.*
- *The sender has no idea if the frame or the acknowledgment is lost, so after the time-out, it resends frame 0, which is acknowledged.*

Flow diagram for Example



In a Stop-and-Wait ARQ system, the bandwidth of the line is 1 Mbps, and 1 bit takes 20 ms to make a round trip. What is the bandwidth-delay product? If the system data frames are 1000 bits in length, what is the utilization percentage of the link?

Solution

The bandwidth-delay product is

$$(1 \times 10^6) \times (20 \times 10^{-3}) = 20{,}000 \text{ bits}$$

- *The system can send 20,000 bits during the time it takes for the data to go from the sender to the receiver and then back again.*
- *The system sends only 1000 bits.*
- *The link utilization is only 1000/20,000, or 5 percent.*
- *For this reason, for a link with a high bandwidth or long delay, the use of Stop-and-Wait ARQ wastes the capacity of the link.*

What is the utilization percentage of the link in previous Example if we have a protocol that can send up to 15 frames before stopping and worrying about the acknowledgments?

Solution

- *The bandwidth-delay product is still 20,000 bits.*
- *The system can send up to 15 frames or 15,000 bits during a round trip.*
- *This means the utilization is 15,000/20,000, or 75 percent. If there are damaged frames, the utilization percentage is much less because frames have to be resent.*

CONTD...

After each frame sent the host must wait for an ACK

- *Inefficient use of bandwidth*

To improve efficiency, ACK should be sent after multiple frames

Alternatives: Sliding Window protocol

- ✓ *Go-back-N ARQ*
- ✓ *Selective Repeat ARQ*

Pipelining

Pipelining: A task is begun before the previous task has ended

- *There is no pipelining in stop and wait ARQ because we need to wait for a frame to reach the destination and be acknowledged before the next frame can be sent*
- *Pipelining improves the efficiency of the transmission*

Sliding window protocol

Sliding window protocols apply Pipelining :

- ✓ *Go-Back-N ARQ*
- ✓ *Selective Repeat ARQ*

- ➢ *Sliding window protocols improve the efficiency*
- ➢ *multiple frames should be in transition while waiting for ACK.*

  Let more than one frame to be outstanding.
- ➢ *Outstanding frames: frames sent but not acknowledged*

Contd...

- *We can send up to W frames and keep a copy of these frames(outstanding) until the ACKs arrive.*
- *This procedures requires additional feature to be added :sliding window*

Go-Back-N ARQ

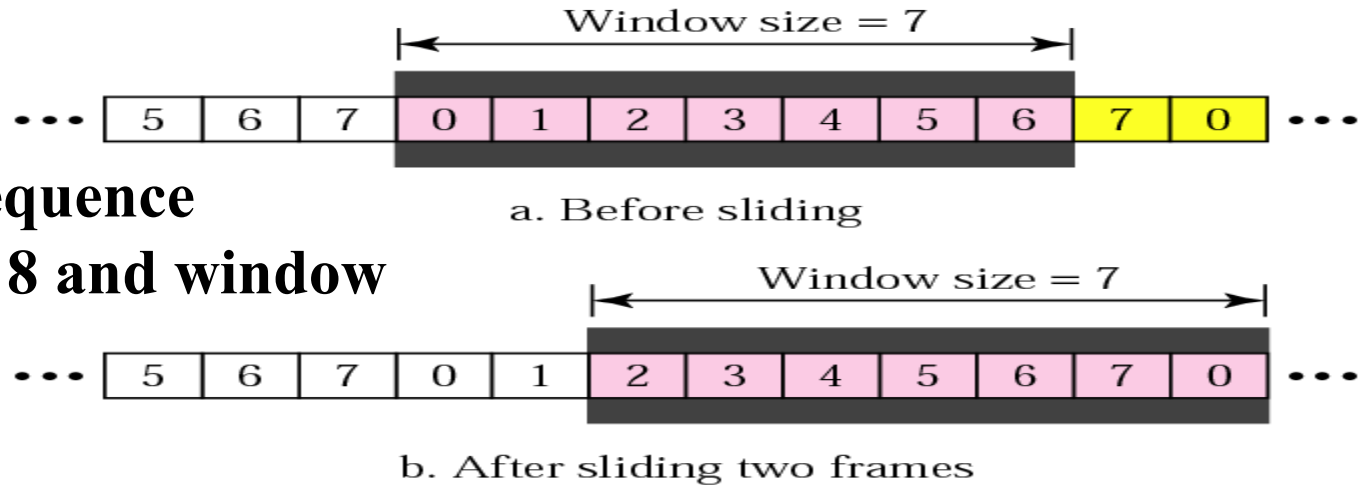
Go_Back_N ARQ

Sender sliding window

- The sender window is an abstract concept defining an imaginary box of size 2 – 1(sequence numbers – 1).
- The sender window can slide one or more slots when a valid acknowledgement arrives.

CONTD...

If m = 3; sequence numbers = 8 and window size =7

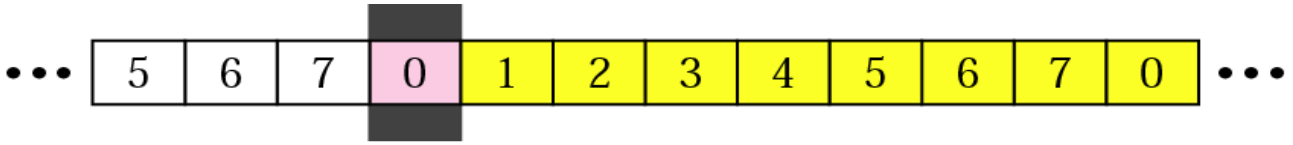


Acknowledged frames

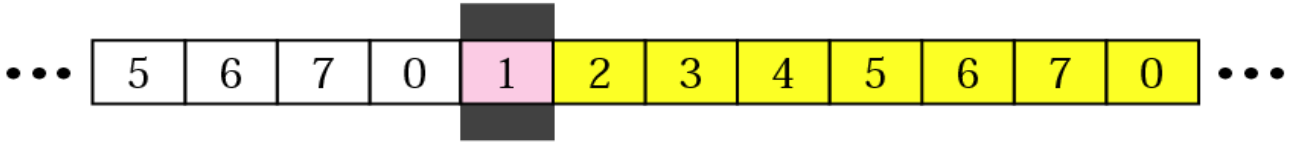
Go_Back _N ARQ

Receiver sliding window

- *The receive window is an abstract concept defining an imaginary box of size 1 with one single variable Rn.*
- *The window slides when a correct frame has arrived; sliding occurs one slot at a time.*

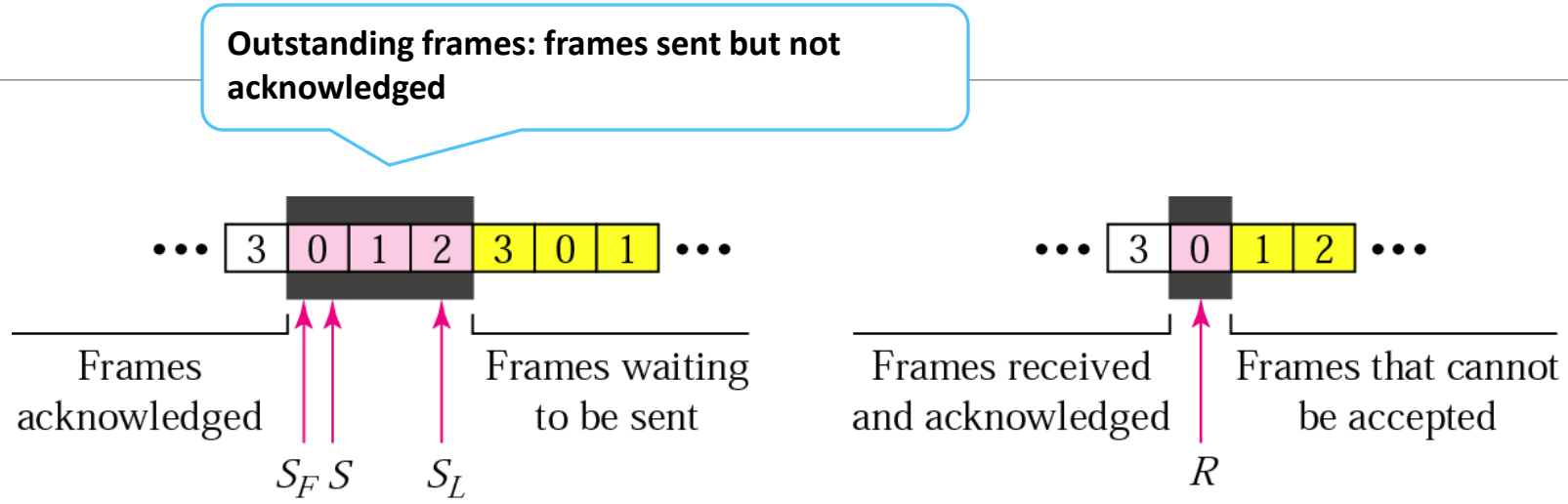


a. Before sliding



b. After sliding

control variables CONTD...



S: hold the sequence number of the recently sent frame

S_F: holds sequence number of the first frame in the window

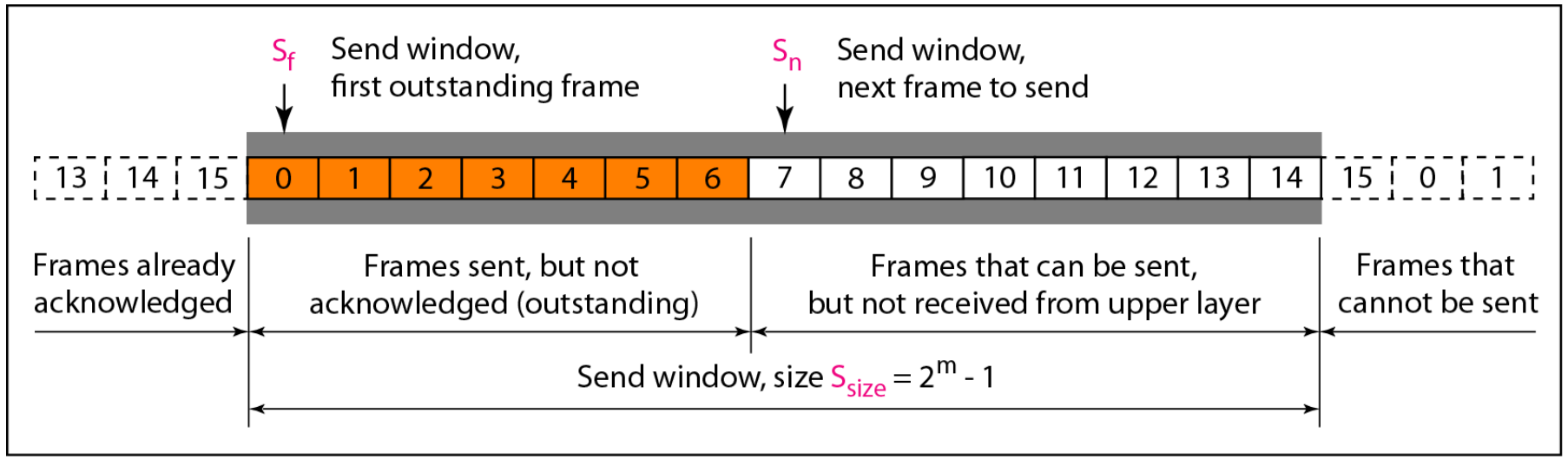
S_L: holds the sequence number of the last frame

R: sequence number of the frame expected to received

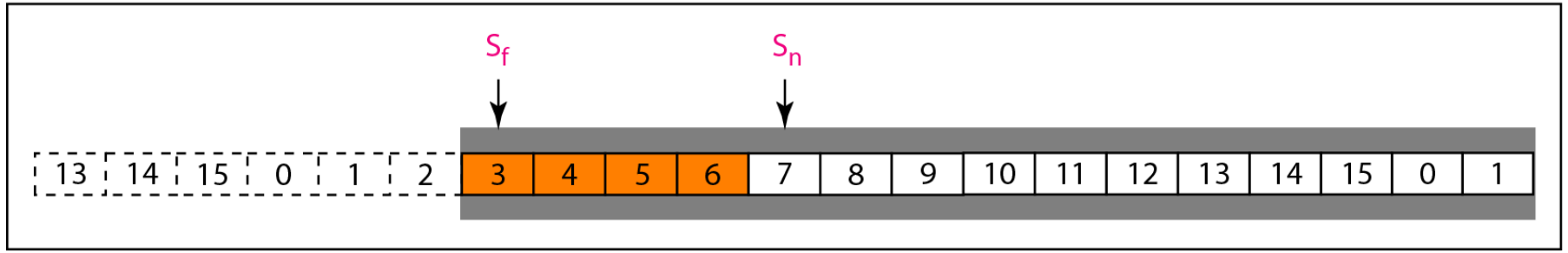
Note

In the Go-Back-N Protocol, the sequence numbers are modulo 2^m, where m is the size of the sequence number field in bits.

Send window for Go-Back-N ARQ



a. Send window before sliding



b. Send window after sliding

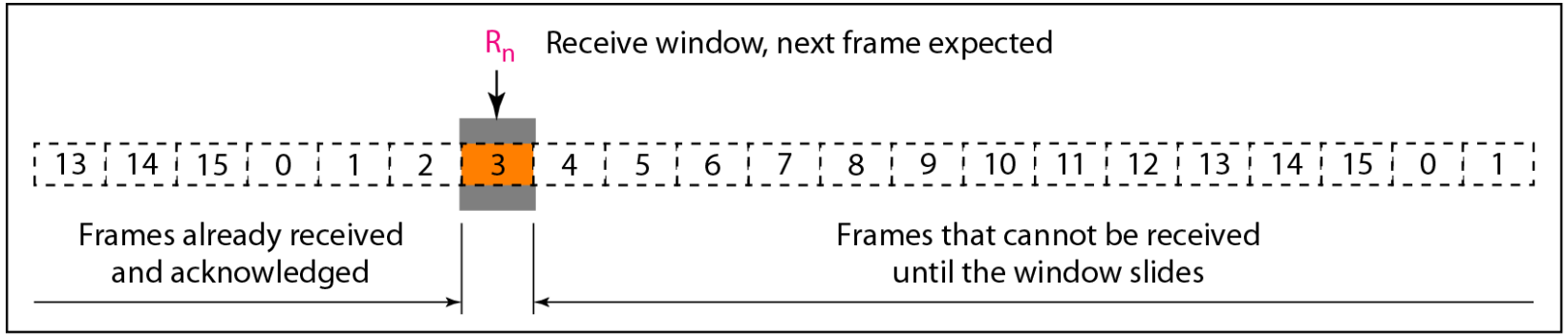
Note

The send window is an abstract concept defining an imaginary box of size $2^m - 1$ with three variables: Sf, Sn, and S_{size}.

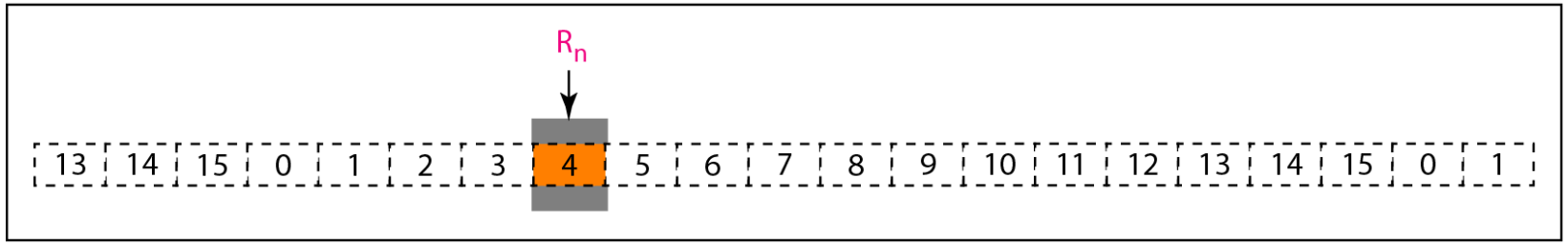
Note

The send window can slide one or more slots when a valid acknowledgment arrives.

Receive window for Go-Back-N ARQ



a. Receive window

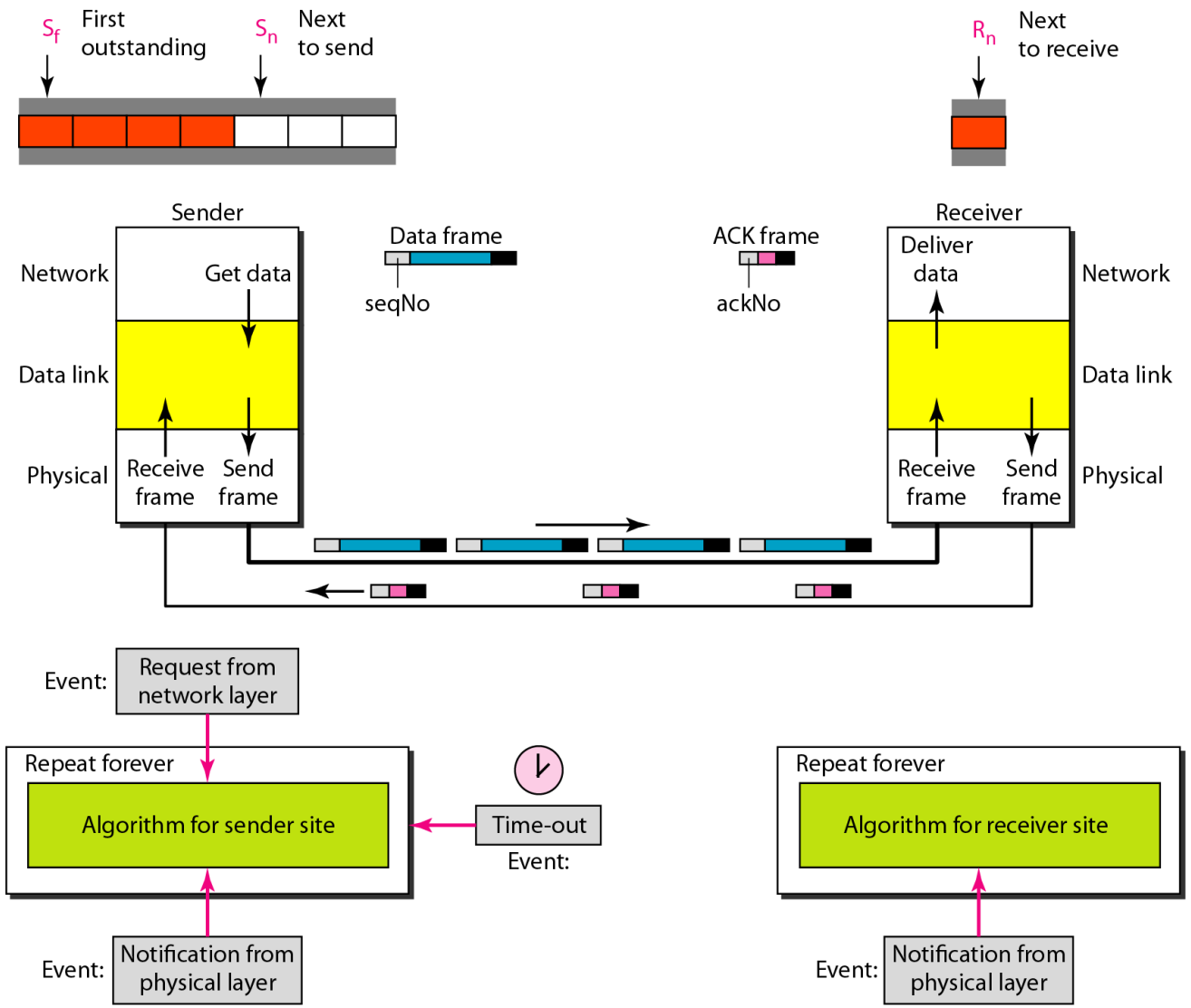


b. Window after sliding

Note

The receive window is an abstract concept defining an imaginary box of size 1 with one single variable R_n. The window slides when a correct frame has arrived; sliding occurs one slot at a time.

Design of Go-Back-N ARQ



Go-Back-N ARQ (Example)



SENDER

RECEIVER



1 Sender sends frames 0 to 3

Window = [0, 1, 2, 3]

0	1	2	3

Frame 0 →

Frame 1 →

Frame 2 →

Frame 3 →

1 Receives 0, 1, 2, 3 (All in order)

0	1	2	3

Sends ACK 4 (Next expected = 4)

← ACK 4

2 ACK 4 received

Window slides

Window = [4, 5, 6, 7]

4	5	6	7

Frame 4 →

Frame 5 →

Frame 6 → ✕ Lost / Error

Frame 7 →

2 Receives 4, 5 (All in order)

4	5

Sends ACK 6 (Next expected = 6)

3 Sender sends frames 4 to 7

Window = [4, 5, 6, 7]

4	5	6	7

← ACK 6

3 Receives 7 (Out of order)

Discards 7

(Still expects 6)

4 Timeout for frame 6 (No ACK 6 received)



Frame 6 (Retransmitted) →

Frame 7 (Retransmitted) →

4 Receives 6, 7 (Both in order now)

6	7

5 Go Back and Retransmit

Resend frames 6 and 7

6	7

← ACK 0

Sends ACK 0 (Next expected = 0)

6 ACK 0 received

Window slides

Window = [0, 1, 2, 3]

0	1	2	3

LEGEND

→ Data Frame (Sender → Receiver)

→ ACK Frame (Receiver → Sender)

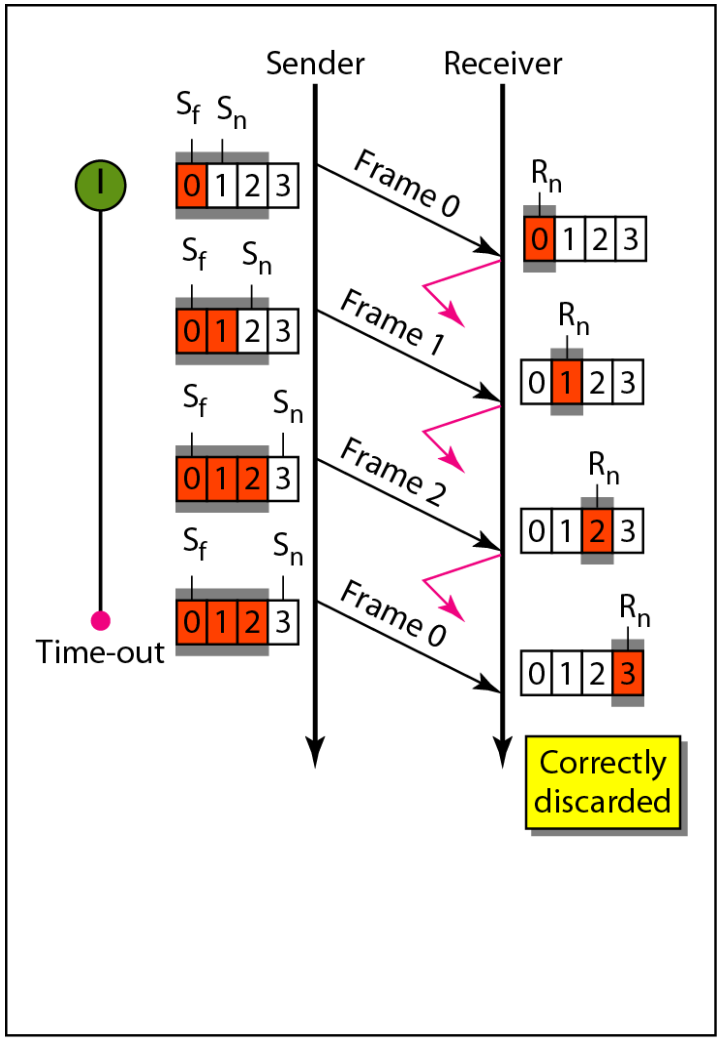
⇢ Retransmission (After Timeout)

X Sequence Number (Example: 3 bits)

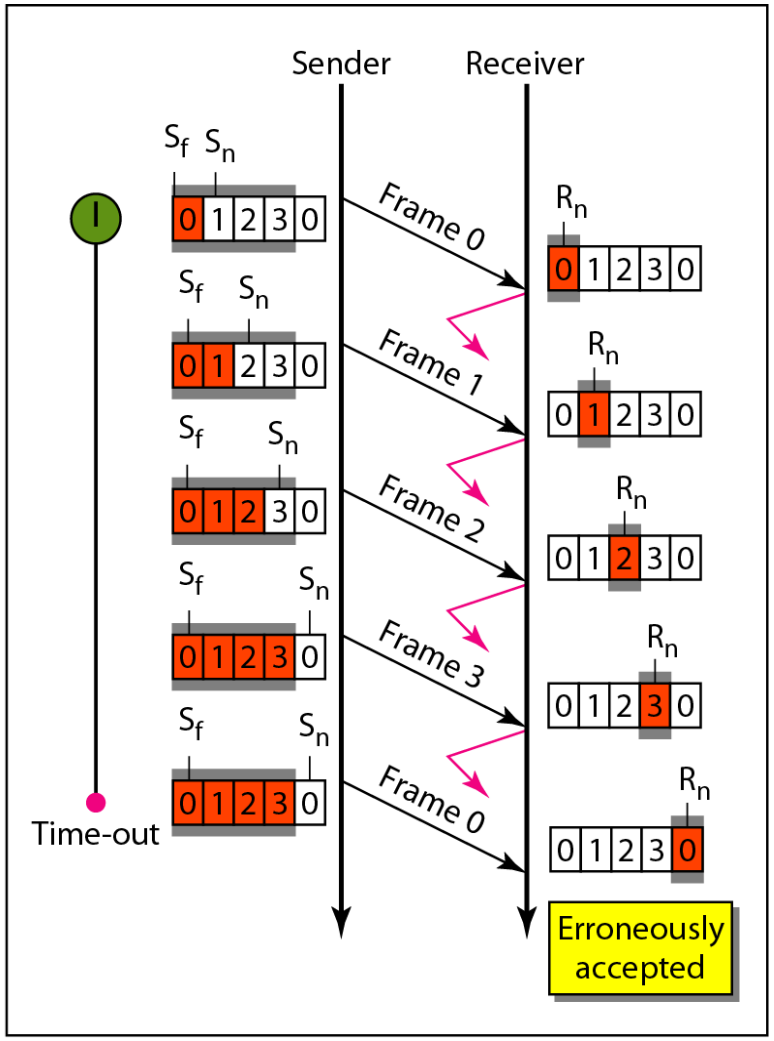
Assumptions

- Window Size (N) = 4
- Sequence Numbers = 0 to 7 (3 bits)

Window size for Go-Back-N ARQ



a. Window size < 2^m



b. Window size = 2^m

Note

In Go-Back-N ARQ, the size of the send window must be less than 2^m; the size of the receiver window is always 1.

Figure shows an example of Go-Back-N.

- *This is an example of a case where the forward channel is reliable, but the reverse is not.*
- *No data frames are lost, but some ACKs are delayed and one is lost.*
- *The example also shows how cumulative acknowledgments can help if acknowledgments are delayed or lost.*
- *After initialization, there are seven sender events.*

- *Request events are triggered by data from the network layer; arrival events are triggered by acknowledgments from the physical layer.*
- *There is no time-out event here because all outstanding frames are acknowledged before the timer expires.*
- *Although ACK 2 is lost, ACK 3 serves as both ACK 2 and ACK 3.*

Flow diagram for Example

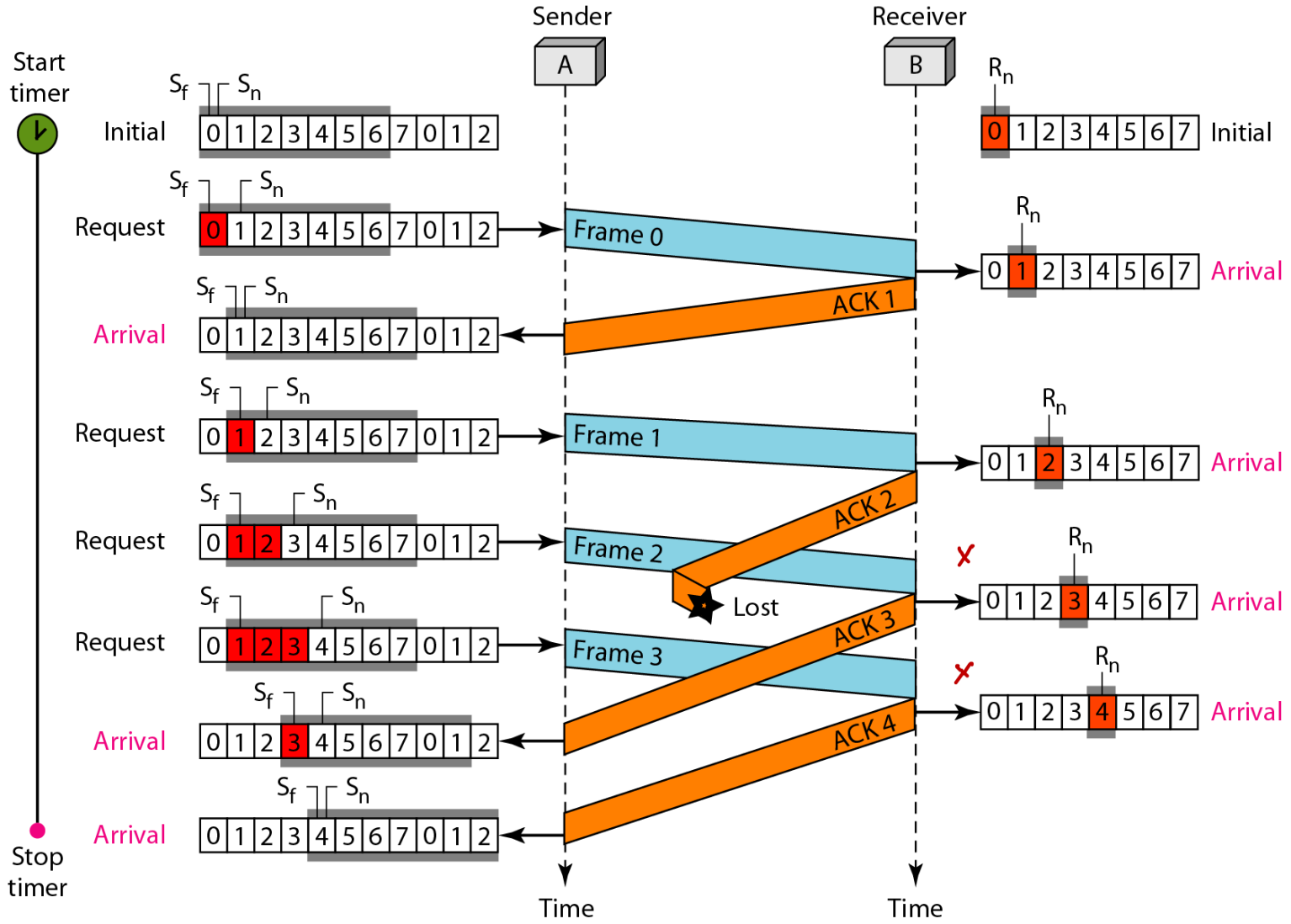


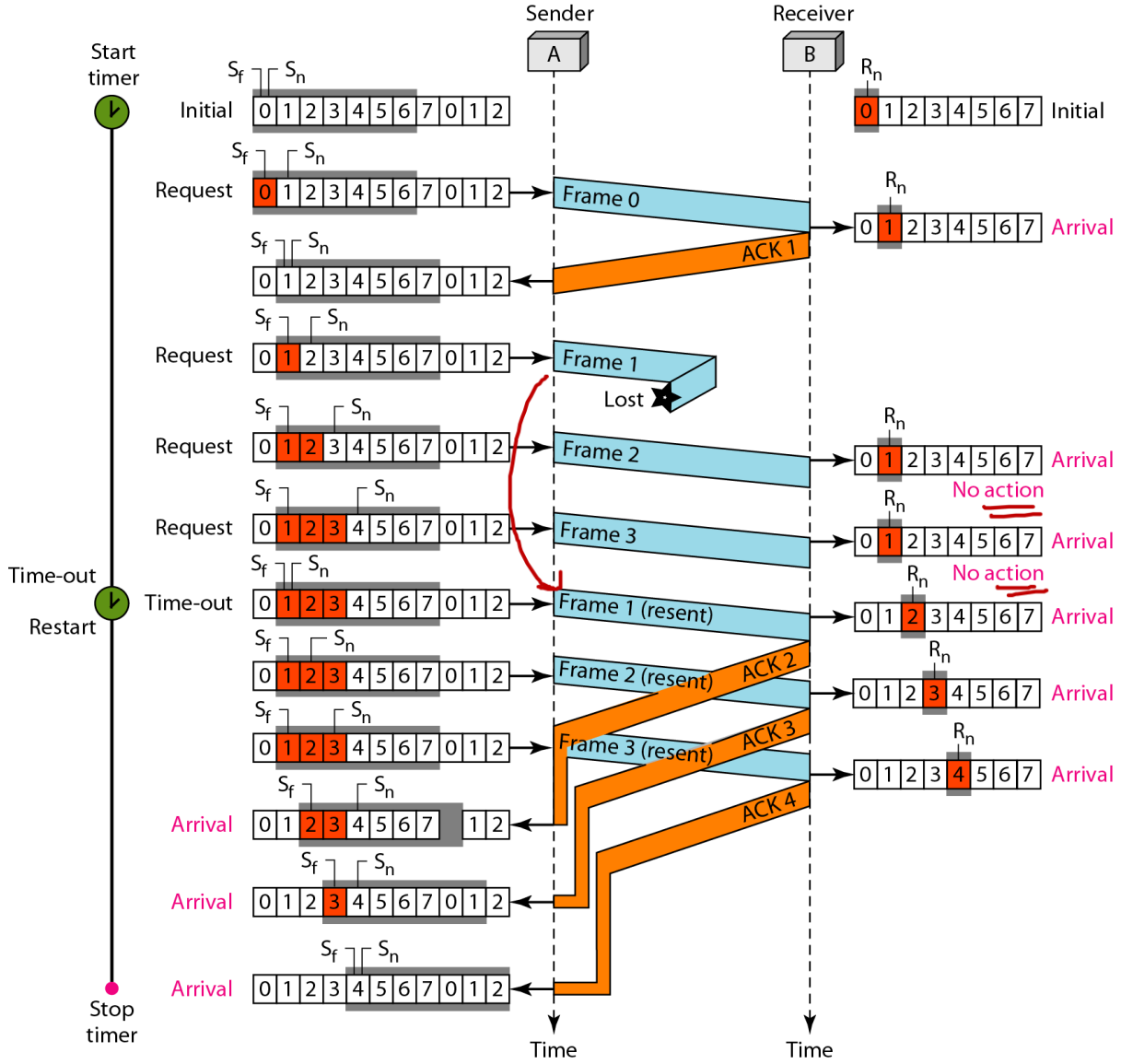
Figure shows what happens when a frame is lost. Frames 0, 1, 2, and 3 are sent.

- *Frame 1 is lost. The receiver receives frames 2 and 3, but they are discarded because they are received out of order.*
- *The sender receives no acknowledgment about frames 1, 2, or 3. Its timer finally expires.*
- *The sender sends all outstanding frames (1, 2, and 3) because it does not know what is wrong.*

- *The resending of frames 1, 2, and 3 is the response to one single event.*
- *When the sender is responding to this event, it cannot accept the triggering of other events.*
- *This means that when ACK 2 arrives, the sender is still busy with sending frame 3.*

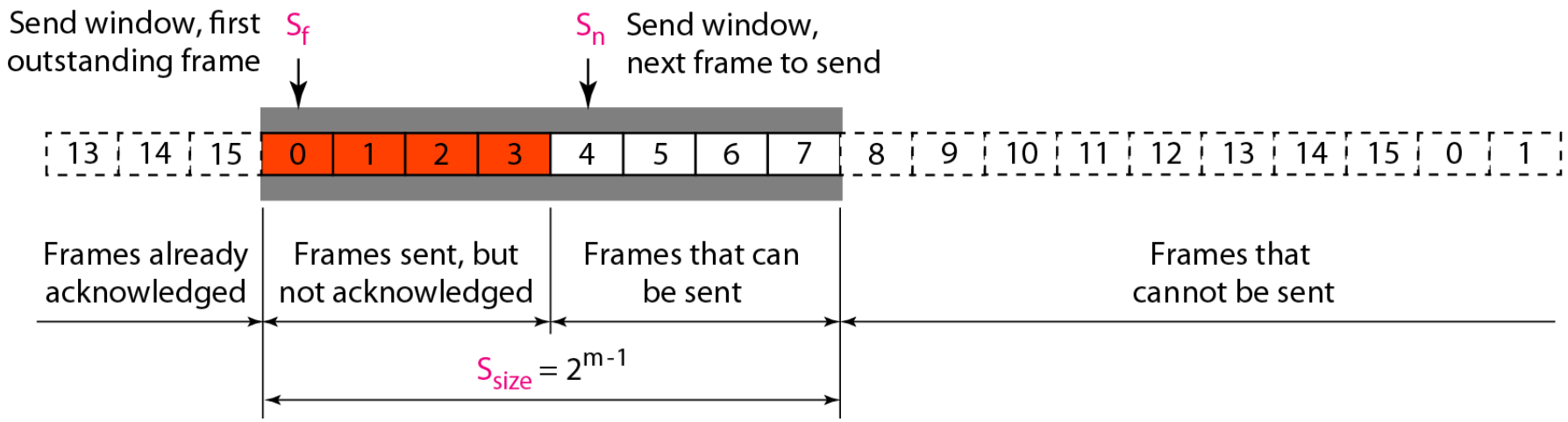
- *The physical layer must wait until this event is completed and the data link layer goes back to its sleeping state.*
- *vertical line to indicate the delay.*
- *It is the same story with ACK 3; but when ACK 3 arrives, the sender is busy responding to ACK 2.*
- *It happens again when ACK 4 arrives.*
- *Before the second timer expires, all outstanding frames have been sent and the timer is stopped.*

Flow diagram for Example

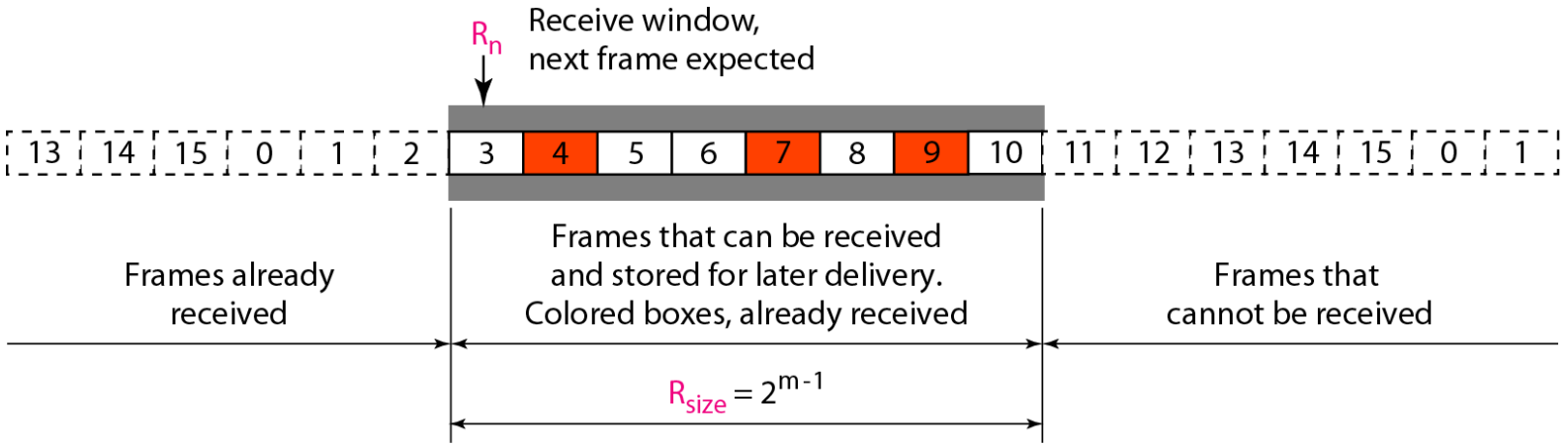


Selective Repeat ARQ

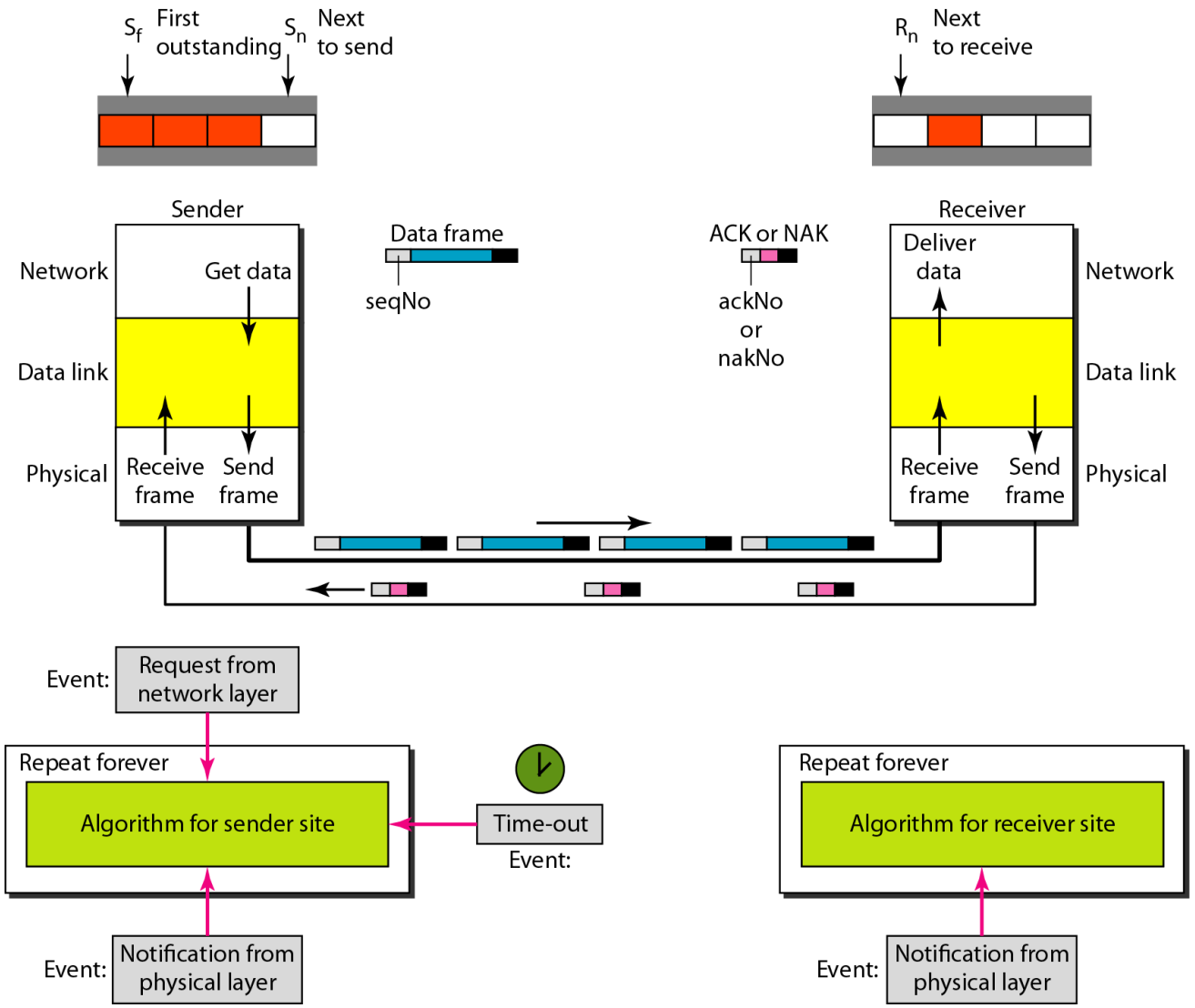
Send window for Selective Repeat ARQ



Receive window for Selective Repeat ARQ



Design of Selective Repeat ARQ



Selective Repeat ARQ (Example)

Window Size (N) = 4
Sequence Numbers: 0 to 7 (3 bits)

SENDER

1 Sender sends frames 0 to 3
Window = [0, 1, 2, 3]
0 1 2 3

2 ACK 0 and ACK 1 received
Window slides
Window = [2, 3, 4, 5]
2 3 4 5

3 Sender sends frames 4 and 5
Window = [2, 3, 4, 5]
2 3 4 5

4 Frame 6 sent but **LOST**
Sender sends frame 7
Window = [4, 5, 6, 7]
4 5 6 7

5 ACK 4, ACK 5 and ACK 7 received
Window slides (6 still missing)
Window = [6, 7, 0, 1]
6 7 0 1

6 Timeout for frame 6
Retransmit frame 6
6

7 ACK 6 received
Window slides
Window = [0, 1, 2, 3]
0 1 2 3

Data Frames (Sender → Receiver)

Frame 0 →
Frame 1 →
Frame 2 →
Frame 3 →

← ACK 0
← ACK 1
← ACK 2
← ACK 3

Frame 4 →
Frame 5 →

Frame 6 → ✕ Lost
Frame 7 →

← ACK 4
← ACK 5
← ACK 7

Timeout for Frame 6
(No ACK 6 received)

Retransmit Frame 6 →

← ACK 6

RECEIVER

1 Receives 0, 1, 2, 3
(All correct, in order)
0 1 2 3
Sends ACK 0 | ACK 1 | ACK 2 | ACK 3

2 Receives 4, 5 (in order)
4 5
Sends ACK 4 | ACK 5

3 Receives 7 (Out of order)
Buffer it 7
Sends ACK 7

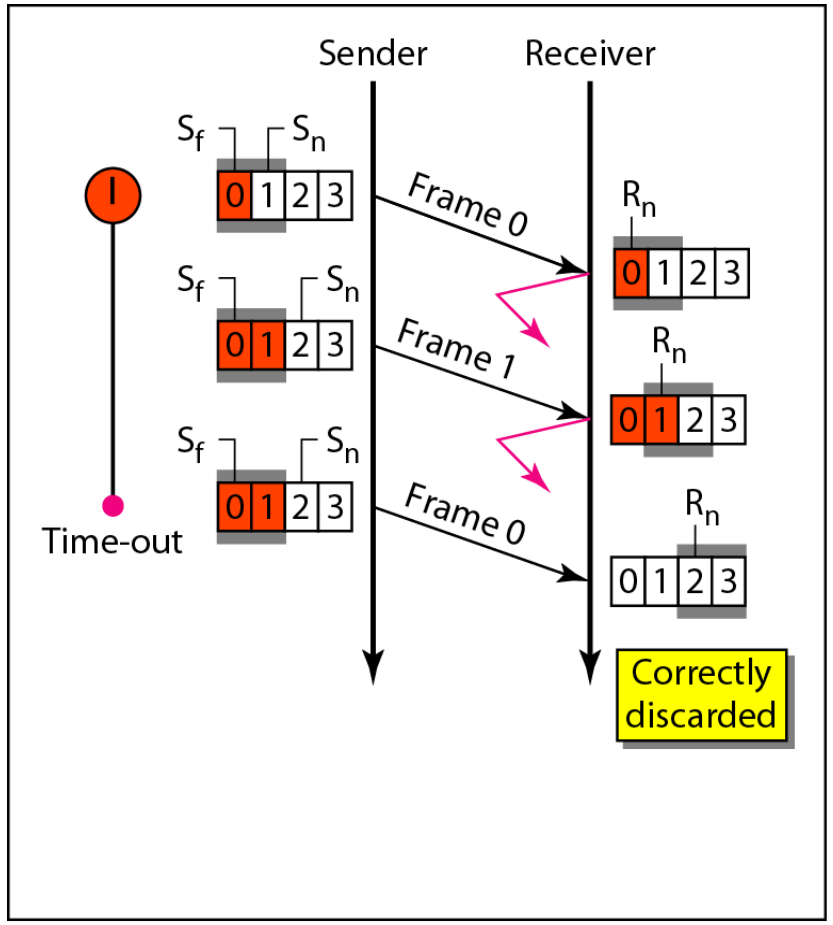
4 Receives 6 (Now in order)
Now have 6 and buffered 7
Deliver 6, then 7
6 7
Sends ACK 6

5 ACK 6 received
All frames up to 7 acknowledged
(Window can slide)
0 1 2 3 4 5 6 7

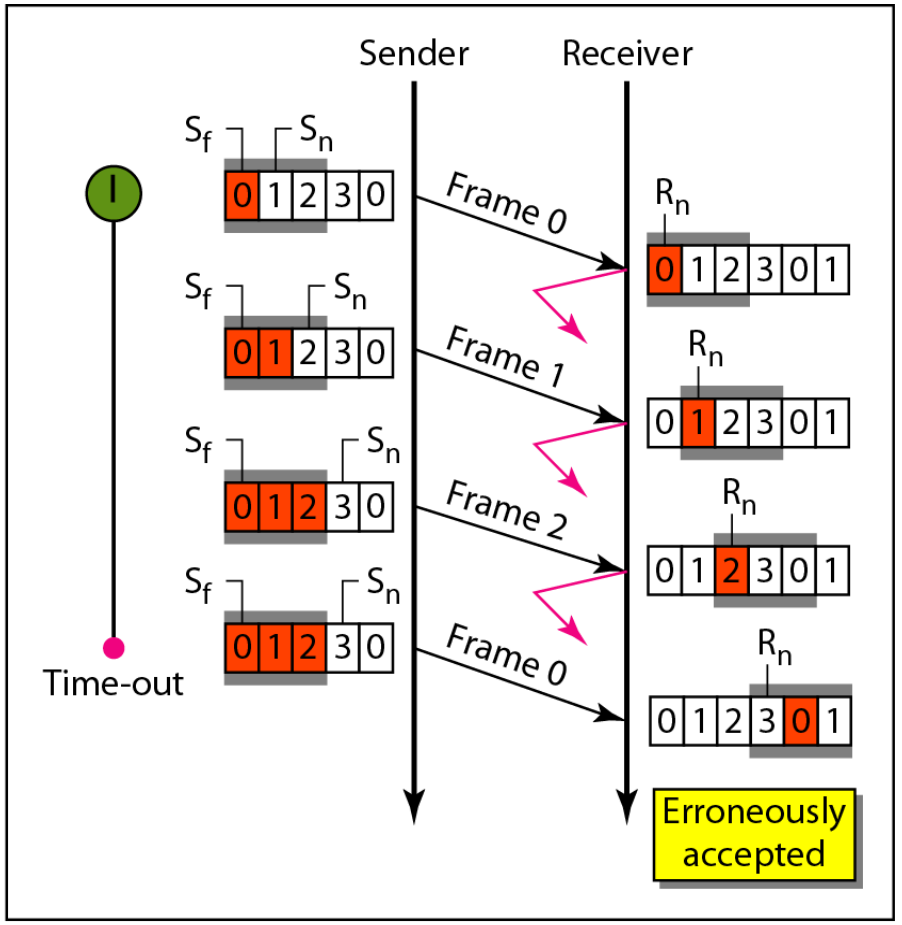
LEGEND

→ Data Frame (Sender → Receiver)
→ ACK Frame (Receiver → Sender)
→ Retransmission (After Timeout)
ACK k — Acknowledgement for Frame k
X — Frame Lost

Selective Repeat ARQ, window size



a. Window size = 2^{m-1}

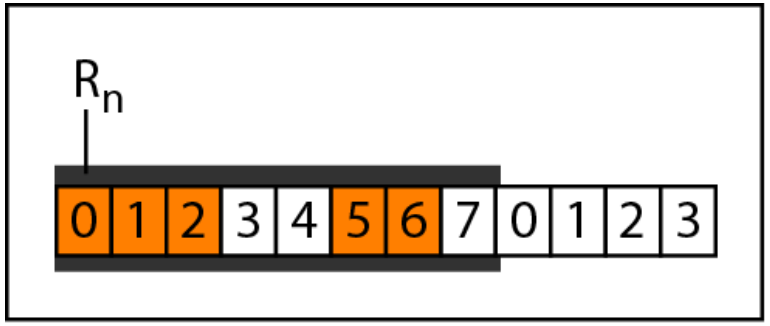


b. Window size > 2^{m-1}

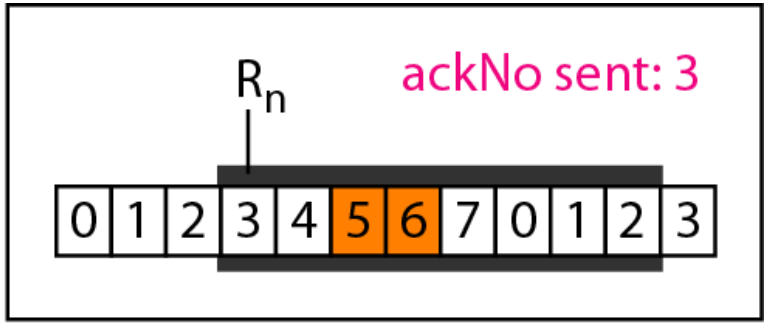
Note

In Selective Repeat ARQ, the size of the sender and receiver window must be at most one-half of 2^m.

Delivery of data in Selective Repeat ARQ



a. Before delivery



b. After delivery

- *One main difference is the number of timers.*
- *Each frame sent or resent needs a timer, which means that the timers need to be numbered (0, 1, 2, and 3).*
- *The timer for frame 0 starts at the first request, but stops when the ACK for this frame arrives.*
- *The timer for frame 1 starts at the second request, restarts when a NAK arrives, and finally stops when the last ACK arrives.*

- *The other two timers start when the corresponding frames are sent and stop at the last arrival event.*
- *At the receiver site we need to distinguish between the acceptance of a frame and its delivery to the network layer.*
- *At the second arrival, frame 2 arrives and is stored and marked, but it cannot be delivered because frame 1 is missing.*

- *At the next arrival, frame 3 arrives and is marked and stored, but still none of the frames can be delivered.*
- *Only at the last arrival, when finally a copy of frame 1 arrives, can frames 1, 2, and 3 be delivered to the network layer.*
- *There are two conditions for the delivery of frames to the network layer: First, a set of consecutive frames must have arrived.*
- *Second, the set starts from the beginning of the window.*

- *Another important point is that a NAK is sent after the second arrival, but not after the third, although both situations look the same.*
- *The reason is that the protocol does not want to crowd the network with unnecessary NAKs and unnecessary resent frames.*
- *The second NAK would still be NAK1 to inform the sender to resend frame 1 again; this has already been done.*

- *The first NAK sent is remembered (using the nakSent variable) and is not sent again until the frame slides.*
- *A NAK is sent once for each window position and defines the first slot in the window.*
- *The first one acknowledges only the first frame; the second one acknowledges three frames.*
- *In Selective Repeat, ACKs are sent when data are delivered to the network layer.*
- *If the data belonging to n frames are delivered in one shot, only one ACK is sent for all of them.*

Flow diagram for Example

